PORTABLE HIGH-PERFORMANCE IMPLEMENTATION OF THE MAXIMUM FLOW PROBLEM USING KOKKOS

Bc. Radek Cichra

Faculty of Information Technology
Czech Technical University in Prague
Department of Theoretical Computer Science
Study program: Informatics
Specialisation: System Programming
Supervisor: doc. RNDr. Dušan Knop, Ph.D.
May 6, 2026

FACULTY
OF INFORMATION
TECHNOLOGY
CTU IN PRAGUE

Assignment of master's thesis

Title:	Portable High-Performance Implementation of the Maximum Flow Problem using Kokkos
Student:	Bc. Radek Cichra
Supervisor:	doc. RNDr. Dušan Knop, Ph.D.
Study program:	Informatics
Branch / specialization:	System Programming
Department:	Department of Theoretical Computer Science
Validity:	until the end of summer semester 2026/2027

Instructions

Graph algorithms are fundamental to solving complex problems in diverse domains, yet their practical implementation on modern heterogeneous hardware presents significant challenges due to irregular memory access patterns and data-dependent control flow.

The goal of this thesis is to investigate the feasibility and performance characteristics of implementing such irregular algorithms using modern Performance Portability frameworks. The work will focus specifically on the Maximum Flow problem and the Push-Relabel algorithm family. Unlike traditional approaches that require maintaining separate codebases for CPUs and GPUs, this thesis aims to utilize the Kokkos C++ library to create a single, device-agnostic implementation.

The student will implement a synchronous, bulk-parallel variant of the Push-Relabel algorithm (PRSN), tailored to map efficiently to the throughput-oriented architecture of GPUs while remaining executable on multi-core CPUs. The work will conclude with a comparative study analyzing the trade-offs between high-level abstraction and raw hardware performance.

Objectives:
1. Theoretical Analysis: Review state-of-the-art parallel approaches for the Maximum Flow problem, with a specific focus on synchronous bulk-parallel variants suitable for GPU execution (e.g., Baumstark et al., 2015).

Electronically approved by doc. RNDr. Dušan Knop, Ph.D. on 21 January 2026 in Prague.

FACULTY
OF INFORMATION
TECHNOLOGY
CTU IN PRAGUE

2. Implementation: Implement the Synchronous Parallel Push-Relabel (PRSN) algorithm using the Kokkos ecosystem. The implementation must be designed to compile for multiple backends (e.g., CPU via OpenMP/Threads, GPU via CUDA/HIP) from a single source codebase.
3. Performance Evaluation: Benchmark the implementation on a representative set of large-scale graphs (e.g., real-world sparse matrices, small-world networks) to evaluate scalability and throughput.
4. Comparative Study (optional): Analyze the "abstraction penalty" (portability tax) and performance portability by comparing execution times and memory bandwidth utilization across different hardware architectures and, where feasible, against available reference implementations.

Electronically approved by doc. RNDr. Dušan Knop, Ph.D. on 21 January 2026 in Prague.

Czech Technical University in Prague
Faculty of Information Technology

I would like to sincerely thank my supervisor, Dušan Knop, for his expertise, insight, guidance, and unwavering positivity. I am also grateful to Jan Pokorný for providing access to the CTU hardware utilized in this thesis. Last but not least, I want to wholeheartedly thank my family and close ones for their support, not only during the writing of this thesis but throughout my entire studies.

Declaration

I hereby declare that the presented thesis is my own work and that I have cited all sources of information in accordance with the Guideline for adhering to ethical principles when elaborating an academic final thesis.

I acknowledge that my thesis is subject to the rights and obligations stipulated by the Act No. 121/2000 Coll., the Copyright Act, as amended. In accordance with Section 2373(2) of Act No. 89/2012 Coll., the Civil Code, as amended, I hereby grant a non-exclusive authorization (licence) to utilize this thesis, including all computer programs that are part of it or attached to it and all documentation thereof (hereinafter collectively referred to as the "Work"), to any and all persons who wish to use the Work. Such persons are entitled to use the Work in any manner that does not diminish the value of the Work and for any purpose (including use for profit) but must preserve the validity of the copyleft licence on which the Work was based. This authorisation is unlimited in time, territory and quantity.

In Prague on May 6, 2026

Abstract

Graph algorithms present significant challenges for modern heterogeneous hardware due to irregular memory access patterns. Historically, achieving high performance on both multi-core CPUs and GPUs has required maintaining fragmented, architecture-specific codebases. This thesis investigates the feasibility of bridging this gap using modern Performance Portability Frameworks. We present the Kokkos Network Flow Solver (KNFS), a single, device-agnostic adaptation of the Synchronous Parallel Push-Relabel (PRSN) algorithm for the Maximum Flow problem, utilizing the Kokkos C++ ecosystem. KNFS maps efficiently to the throughput-oriented architecture of GPUs while remaining executable on CPUs. We evaluated the solver against reference implementations across a diverse benchmark suite of 108 instances. The results demonstrate that KNFS maintains empirical correctness and consistency — outperforming certain reference solvers that exhibited non-deterministic behavior — while delivering competitive execution times. Furthermore, we quantify the framework overhead — measured as the upper bound of execution time spent outside computational kernels — to be roughly 26%. Ultimately, we aim to demonstrate that true performance portability is an achievable reality; developers can utilize a single, highly maintainable codebase to solve even notoriously hostile, irregular graph problems without sacrificing critical hardware performance.

Keywords C++, GPU Acceleration, Graph Algorithms, High-Performance Computing, Kokkos, Maximum Flow Problem, Performance Portability, Push-Relabel Algorithm

Abstrakt

Grafové algoritmy představují pro moderní heterogenní hardware značnou výzvu kvůli nepravidelnému přístupu do paměti. Dosažení vysokého výkonu jak na vícejádrových procesorech (CPU), tak na grafických akcelerátorech (GPU), historicky vyžadovalo udržování několika alternativ kódu specifických pro danou architekturu. Tato diplomová práce zkoumá možnosti adaptace moderních frameworků pro přenositelnost výkonu (Performance Portability Frameworks). Výstupem je Kokkos Network Flow Solver (KNFS), hardwarově agnostická adaptace synchronního paralelního Push-Relabel (PRSN) algoritmu pro problém maximálního toku, využívající C++ ekosystém Kokkos. KNFS se efektivně mapuje na architekturu GPU orientovanou na propustnost, přičemž zůstává spustitelný i na CPU. KNFS jsme vyhodnotili v porovnání s referenčními implementacemi na rozmanité sadě čítající 108 instancí. Výsledky ukazují, že si KNFS zachovává empirickou správnost a konzistenci — čímž překonává některé referenční řešiče, které vykazovaly nedeterministické chování — a zároveň dosahuje konkurenceschopného výkonu. Dále kvantifikujeme režii frameworku — měřenou jako horní mez doby běhu strávené mimo výpočetní kernely — na zhruba 26%. Naším cílem je ukázat, že skutečná přenositelnost výkonu je dosažitelnou realitou; vývojáři mohou využít jedinou, vysoce udržovatelnou implementaci k řešení i notoricky problematických, nepravidelných grafových problémů, aniž by obětovali kritický hardwarový výkon.

Klíčová slova Algoritmus Push-Relabel, C++, GPU akcelerace, Grafové algoritmy, Kokkos, Přenositelnost výkonu, Problém maximálního toku, Vysoce výkonné počítání

Contents

List of Figures

List of Tables

Introduction

Graphs and graph theory are, without a doubt, crucial parts of modern computer science and many other (even seemingly unrelated) fields. This is because graphs are the fundamental language of complex systems, capturing and modeling almost any problem. This makes them extremely useful, as they allow for solving many problems from diverse domains in a unified manner. If a challenge can be reduced to a known graph problem or its variation, it can often inherit efficient, proven solutions from it. It is this universality that makes graph theory a well-studied and always evolving field.

While in theory, this sounds like an ideal approach, there are challenges regarding practical implementation on modern hardware. Unlike dense matrix operations or grid-based simulations, graph algorithms heavily rely on irregular memory access stemming from algorithm logic built on traversing and working *along edges*. Such traversal often requires *pointer chasing* across non-contiguous memory locations, which degrades cache utilization on CPUs and hinders throughput performance on GPUs.

The landscape of High Performance Computing has become increasingly heterogeneous in terms of device architectures, as both CPUs and GPUs are utilized, often in tandem. However, there is little to no similarity between designs that are performant for one or the other due to architectural differences. Historically, this has led to shattered portability and the need to maintain multiple codebases, resulting in a great deal of duplicate effort and wasted man-hours. This warrants a shift from device-specific optimization to performance portability — the ability to express a single algorithmic definition that, *in theory*, compiles to efficient native machine code on multiple architectures.

In this thesis, we tackle these challenges by adapting a **Synchronous Parallel Push-Relabel (PRSN)** algorithm for finding maximum flow using **Kokkos**, an established C++ Performance Portability Ecosystem. PRSN was selected due to its utilization of a bulk-synchronous parallel model, which we believe maps well onto GPUs, as it separates local computation from global updates.

The culmination of this work is the **Kokkos Network Flow Solver**

(KNFS), a single device-agnostic codebase capable of executing on both multi-core CPUs and GPUs. The thesis itself is structured as follows:

In chapters 1 and 2, we establish the theoretical foundation of the Maximum Flow problem and the algorithms relevant to our work, followed by an analysis of GPU architectures and the introduction of the Kokkos framework model.

In chapters 3 and 4, we review the current landscape and introduce four reference implementations of varying approaches and target architectures to compare against. We then detail the complete pipeline design of our KNFS solver, from parallel graph loading to the core computational kernels, as well as an experimental degree-based hybrid modification.

Finally, in chapters 5 and 6, we describe and comment on our experimental evaluation process. This includes a comparative study of execution times, correctness, and a direct analysis of Kokkos utilization. We conclude by discussing observed issues, such as flow sloshing, and outlining potential avenues for future optimization.

In essence, this work serves as a stress test for performance portability. By successfully maintaining a viable baseline of performance and empirical correctness on irregular graph algorithms — a class of problems notoriously hostile to GPU architectures — we demonstrate that the substantial benefits of a single, highly maintainable codebase do not require an unacceptable sacrifice in speed.

Chapter 1

Max Flow Problem & Solutions

1.1 The Maximum Flow Problem

1.1.1 Definition

A **flow network** is defined as a directed graph $G = (V, E)$, where V is a set of vertices and E is a set of directed edges. Within this network, we note two special nodes: a **source** node $s \in V$ and a **sink** node $t \in V$, such that $s \neq t$.

Each edge $(u, v) \in E$ is assigned a non-negative **capacity**, defined by a capacity function $c : V \times V \to \mathbb{R}_{\geq 0}$. If there is no edge between u and v, the capacity is assumed to be $c(u, v) = 0$.

A **flow** in this network is a function $f : V \times V \to \mathbb{R}$ that assigns a real number to each pair of nodes, **subject to two constraints** in order to partially model the physical properties of real flow:

- **Capacity Constraint:** The flow along any edge must be non-negative and cannot exceed the edge's given capacity.

$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in V \times V$$

- **Flow Conservation:** For every **common node** (not s and t) in the network, the total flow entering the node must exactly equal the total flow exiting the node.

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v) \quad \forall u \in V \setminus \{s, t\}$$

The **value of a flow**, denoted as $|f|$, represents the net amount of flow leaving the source node, which, due to the flow conservation constraint, must

equal the net flow entering the sink node:

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$$

The **Maximum Flow Problem** is the optimization problem of finding a flow function f that maximizes the overall flow value $|f|$ for a given network G, source s, sink t, and capacity function c.

The problem can also be restated as the **Min Cut Problem**, as these problems are linked by a strong dual relationship, thus leading to the same result [1].

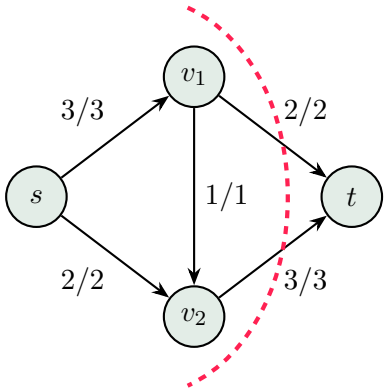


Figure 1.1 Mock flow network displaying a solved maximum flow ($|f| = 5$) and the corresponding minimum cut separating $\{s, v_1, v_2\}$ from $\{t\}$.

1.1.2 Motivation & Relevance

The Max Flow Problem was first formally described more than 70 years ago in a then classified Cold War document, serving the purpose of modeling Soviet rail network logistics [2].

As time went on, it became rather clear that many problems can be reduced to and treated as a Max Flow Problem or its variations. This made it an indispensable and foundational tool in many areas, both research and industry alike. Often listed uses include those in areas of **Transportation & Logistics** or **Digital Networks & Communications**. There are, however, many more areas, such as **Medical Imaging**, **Load Balancing**, and **Sport Elimination** [3][4][5].

Because of its immense versatility and usefulness in both theoretical and practical fields, it is still an actively studied topic, with work being put towards new solution approaches [6].

1.2 The Push-Relabel Algorithm

1.2.1 Concept & Preflows

Unlike augmenting path algorithms (such as Ford-Fulkerson [7] or Edmonds-Karp [8]), which maintain flow conservation at all times and search for augmenting paths between the source and sink, the **Push-Relabel** algorithm, introduced by Goldberg and Tarjan, operates under slightly different constraints [9].

It relaxes the flow conservation constraint, allowing nodes to temporarily hold more incoming flow than outgoing flow. This intermediate state is usually called **preflow**. With the notion of preflow, the strict flow conservation constraint is replaced by a relaxed version:

$$\sum_{v \in V} f(v,u) \geq \sum_{v \in V} f(u,v) \quad \forall u \in V \setminus \{s\}$$

The net difference between incoming and outgoing flow at a node u is the **excess flow**, denoted as $e(u)$:

$$e(u) = \sum_{v \in V} f(v,u) - \sum_{v \in V} f(u,v) \geq 0$$

A node $u \in V \setminus \{s,t\}$ is considered **active** if $e(u) > 0$. The algorithm terminates when there are no active nodes left (i.e., all excess flow has been *drained* by returning to the source), at which point the preflow becomes a valid maximum flow.

1.2.2 Residual Network & Height Function

To determine where (excess) flow can be pushed, we define a **residual network** $G_f = (V, E_f)$. For every edge (u,v), the **residual capacity** $c_f(u,v)$ is the amount of additional flow that can be sent from u to v before reaching capacity:

$$c_f(u,v) = c(u,v) - f(u,v) + f(v,u)$$

An edge (u,v) is considered a **residual edge** ($(u,v) \in E_f$) if $c_f(u,v) > 0$.

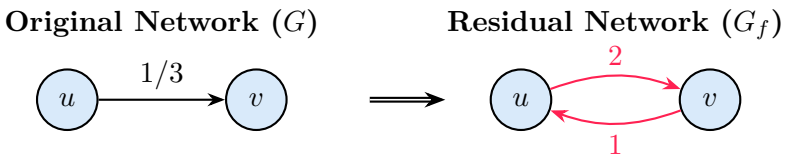


Figure 1.2 Flow Network and its Residual Network at a state, where there is flow of 1 on edge (u,v), leading to $c_f(u,v) = 3 - 1 + 0 = 2$; $c_f(v,u) = 0 - 0 + 1 = 1$

To prevent infinite loops and promote moving flow throughout the graph towards the sink, the algorithm utilizes a **height function** (also called labeling) $h : V \to \mathbb{N} \cup \{0\}$. This function must satisfy the following conditions:

- $h(s) = |V|$
- $h(t) = 0$
- $h(u) \leq h(v) + 1 \quad \forall (u, v) \in E_f$

It is supposed to mirror flow running *downhill* towards the sink, and thus flow can only be pushed from a higher label node to an immediately lower adjacent node. The height function allows for *optimal progress* if it precisely reflects the current distance (in the number of edges in the shortest path) to t in G_f.

1.2.3 Core Operations

The algorithm initializes by pushing as much flow as possible from the source s to all its neighbors, saturating those edges and setting the height of s to $|V|$. This sets s as the *tallest* point from which the flow will traverse the graph towards t. Then, it repeatedly applies two local operations until no active nodes remain:

- **Push:** If an active node u has a residual edge to a neighbor v such that $h(u) = h(v) + 1$, flow is pushed from u to v. The amount of flow pushed is $\min(e(u), c_f(u, v))$. Note that pushing mutates both the original network and its residual network (see figure 1.3).
- **Relabel:** If an active node u cannot push flow (because all neighbors v with $c_f(u, v) > 0$ have $h(v) \geq h(u)$), its height is increased. The new height is set to:

  $$h(u) = 1 + \min\{h(v) \mid (u, v) \in E_f\}$$

  This ensures u will be *uphill* from some neighbor and can push to it in the future.

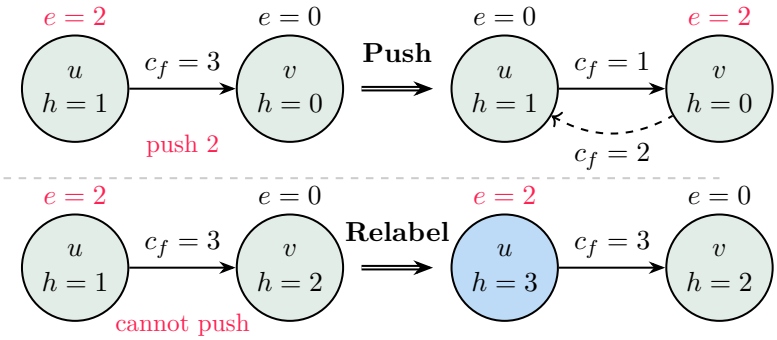


Figure 1.3 Examples showing how local **Push** and **Relabel** operations mutate excess flow (e), node height (h), and residual capacity (c_f).

1.3 Synchronous Bulk-Parallel Execution

1.3.1 Parallel Push-Relabel

Due to the locality of Push-Relabel operations, the approach with relaxed flow constraints described in 1.2 is an attractive candidate for parallelization. In theory, multiple active nodes can easily be processed in tandem by different threads. Problems arise when active nodes are close to each other — at that point, individual operations performed by each active node may clash and cause race conditions.

Consider the two main problematic scenarios:

1. Multiple neighboring nodes attempt to push flow to the same target node v simultaneously
2. A neighbor attempts to read the height v exactly while v is being relabeled.

These scenarios need to be managed, as the race conditions introduced by them can easily break the fundamental constraints of the problem, as seen in figure 1.4.

Dealing with these issues in an asynchronous manner, which is the common way of adapting the Push-Relabel algorithm, requires heavy use of locking or atomic operations when working with the excess flow and height of a given vertex. On the GPU, this contention effectively serializes memory access and likely defeats the purpose of using the GPU in the first place.

1.3.2 The Bulk-Synchronous Parallel Paradigm

To mitigate the issues described in subsection 1.3.1 and map the algorithm efficiently to the GPU, the problem must be adapted using the **Bulk-Synchronous Parallel (BSP)** paradigm [10]. It divides computation into a series of supersteps. Each superstep consists of three phases:

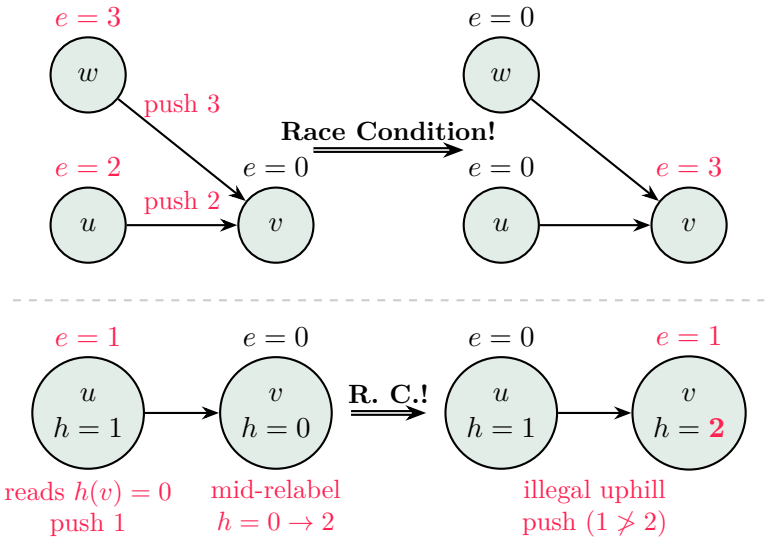


Figure 1.4 Examples showing **a)** potential loss of flow / corruption due to the race condition while pushing and **b)** breaking *downhill* push rule due to race condition while reading labels

1. **Concurrent Computation:** Every thread performs local computations using strictly read-only access to the global state from the previous superstep.
2. **Communication:** Threads exchange data or accumulate updates.
3. **Barrier Synchronization:** All threads wait until the global state is fully updated and consistent before proceeding to the next superstep.

This approach differs from the *common* way of implementing parallel Push-Relabel in that it splits the algorithm into multiple phases with synchronization between them, **but not during** those phases and operations within them. By applying the BSP model to our cause, we arrive at the synchronous parallel Push-Relabel approach [11], upon which this thesis stands and builds. It is described in the following section 1.4.

1.4 The Synchronous Parallel Push-Relabel (PRSN) Algorithm

Synchronous Parallel Push-Relabel (PRSN) algorithm, as described in the works of Baumstark et al. [11], was the base reference for our implementation efforts in the following chapters.

1.4.1 Simple PRSN

The initial phase of saturating edges from the source s is the same as in the normal Push-Relabel. The main loop of the algorithm works by performing four steps while there are still **active** (1.2.1) vertices being found. The steps are:

1. **Process all active vertices in parallel**. This entails checking all outgoing edges and determining if we can legally push (1.2.3) along them. Any possible pushes are performed, and changes to the flow excess are buffered (performed on a copy).
2. **Still active vertices relabel in parallel**. This means any active vertex unable to push all of its flow out in step 1 will relabel (1.2.3) to allow for further discharge later. This relabel is also buffered.
3. **Apply new labels**. Commit to the relabels of the previous step.
4. **Apply new excess**. Commit to the pushes made in step 1.

Occasionally, once an arbitrary work threshold is reached, a **Global Relabel** phase is run, which performs a reverse **BFS** in the Residual Network G_f (1.2.2) from the sink t to align node labels to reflect their **exact** distance from it. This is because relabels in step 2 steer the height of the nodes away from their actual distance, thus slowing algorithm convergence. Note that Global Relabel is just an optimization step (albeit basically a mandatory one for practical use), and the algorithm works without it.

There is a flaw in the four step process above in that it allows for a maximum of just one relabel per algorithm iteration. We would ideally like to fully discharge any active vertices in each algorithm iteration, but that necessitates alternating between steps 1 and 2. We cannot simply allow that, as alternating steps 1 and 2 freely introduces two distinct hazards for adjacent active vertices:

- **Concurrent Data Races:** In a single iteration, adjacent nodes might attempt to relabel and push across the shared edge simultaneously, corrupting the graph state.
- **Flow Sloshing:** Across multiple iterations, nodes might fall into a localized loop-taking turns relabeling just above each other and pushing the

same excess flow back and forth without making global progress toward the sink (see figure 1.5).

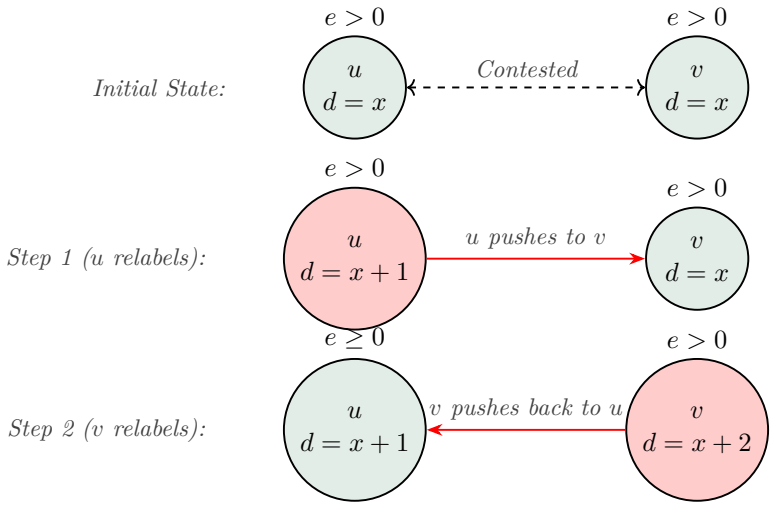


Figure 1.5 Adjacent active vertices u and v can stall flow, continuously relabeling past one another and sloshing excess flow back and forth without making global progress.

While the flow sloshing is mitigated by periodic Global Relabeling (as described in 1.4.1), the immediate threat of concurrent data races requires strict arbitration. To safely resolve these intra-iteration collisions, PRSN introduces a deterministic discharge criterion.

1.4.2 Discharge criterion

PRSN introduces a very powerful deterministic criterion that can be used to decide which vertex *owns* the contested edge for a given iteration. If two adjacent vertices v and w are both active, vertex v wins the edge if it meets any of the following criteria (evaluated in this order):

- $h(v) < h(w) - 1$
- $h(v) = h(w) + 1$
- $v < w$

Crucially, this check **requires no atomic operations or locks**. This is because the algorithm evaluates the criterion using the global distance labels h from the beginning of the iteration, which remain strictly read-only during the discharge phase. Any local relabeling updates during the discharge phase are tracked in a private state (h') to evaluate edge admissibility, but the

deterministic tie-breaker relies strictly on the global, unchanging state of h. Because all threads read from the same unchanging state, they independently and consistently arrive at the exact same conclusion regarding who wins the edge.

Chapter 2

GPU & Performance Portability Framework(s)

2.1 GPU

GPU (Graphics Processing Unit) is a hardware component initially developed for accelerated image rendering and computer graphics. As time has shown, its architecture also makes it exceptionally well-suited for general-purpose parallel computation. Today, GPUs are foundational accelerators in HPC, artificial intelligence, and academia.

2.1.1 Architecture: Latency & Throughput

CPUs are fundamentally latency-oriented. They are heavily optimized for mostly sequential execution, utilizing highly sophisticated tools such as branch prediction and featuring complex multi-layer caches.

In stark contrast to this, GPUs are much more throughput-oriented at the cost of finer control over each individual core they have (see figure 2.1). Having many more cores, though they are comparatively less complex, makes GPUs ideal for executing thousands of threads simultaneously in parallel. To hide memory latency, GPUs utilize the sheer number of threads (or rather, groups of threads, called **warps** by NVIDIA and **wavefronts** by AMD) that are assumed to be active at any given time. The GPU scheduler can switch between these groups so that those ready to run do so, while others are waiting for data [12][13].

2.1.2 GPU Memory Bandwidth & Bottleneck

Note that the GPU, being its own hardware component, has its own onboard memory and memory space separate from the CPU (DRAM and cache memory

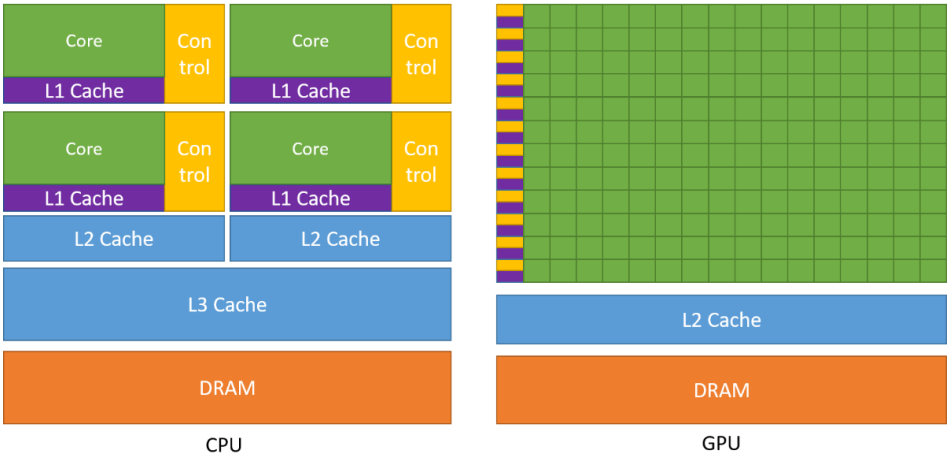


Figure 2.1 The GPU devotes more transistors to data processing [13].

in figure 2.1). To reap the benefits of parallel execution on the GPU, computations must primarily operate within this dedicated memory space. This is because moving data onto the GPU itself is orders of magnitude slower than any local communication or data manipulation within the GPU. The slowdown caused by frequent communication between the CPU and GPU would most likely render GPU usage redundant.

Furthermore, because GPUs can compute so much data simultaneously, their performance is often less constrained by the sheer number of mathematical operations they can perform (compute-bound) and more constrained by how quickly they can fetch data from the onboard DRAM to feed the processing cores (memory bandwidth-bound).

2.1.3 Graph Algorithms on GPU

Trying to perform graph algorithms on the GPU can be challenging due to the inherent nature of graph representation and the way memory is accessed when working with it. Graph edges reflect the information we are trying to model, and working with this information necessitates working along these edges, which generally leads to irregular memory access patterns. While there are fields, such as computer vision, where the problems we are trying to solve have highly regular inputs that can be exploited, this does not hold true in general.

Due to this, traversing a graph often requires fragmented work across non-contiguous memory locations. If multiple threads from the same warp (2.1.1) try to access data from scattered memory addresses, the GPU cannot coalesce these requests into a single cache pull, which hinders its throughput.

2.2 Performance Portability Frameworks

The modern HPC landscape has become increasingly complex, particularly when it comes to hardware layer architecture. This shift from a rather unified, multi-core CPU model to a heterogeneous HPC system with accelerators and coprocessors, often with different architectures (GPU), has been recognized for more than a decade (as in [14]) and has not slowed down since. Today, just over 50% of the Top500 (and 78% of the Top100 [15]) HPC machines use accelerators, the majority of those being GPUs. Moreover, while these heterogeneous machines make up roughly half of the Top500, they account for over 86% of the peak performance of the whole list combined (data as of November 2025 from [16]). This is a clear indicator of where the landscape of HPC is most likely going to move onward.

Due to the high diversity in the architecture and structure of any given HPC cluster or machine, writing complex programs practically warrants multiple codebases for a selected set of target configurations, which leads to many wasted developer hours on redundant work that must be done just to adapt some code to a specific target architecture. A potential solution to this has emerged in the form of Performance Portability Frameworks (**PPFs**).

2.2.1 Definition

A PPF can be described as a programming paradigm designed to achieve portability through the abstraction and detachment of algorithmic logic from any hardware architecture specific dependencies. In tandem with this, it aims to maintain performance that is competitive with solutions tailored for a given hardware architecture.

Rather than writing explicit, low-level directives tied to a specific vendor (such as CUDA `__global__` or OpenMP `#pragma omp for`), the developer expresses algorithms using high-level, potentially parallel patterns such as `parallel_for`, `parallel_reduce`, and `parallel_scan`. It is the PPF's job to then *slot in* specific directives and changes needed to make the code functional on a specific target architecture. PPFs can also abstract data management so that they can map their memory layout to best suit the target architecture at compile time [14][17][18].

This means that, **in theory**, PPFs offer a way to maintain just one **device-agnostic codebase** capable of compiling against multiple backends with different architectures while being runnable and performant on all of them. As the title of this thesis suggests, the main PPF in the context of this work is **Kokkos** (see subsection 2.2.2).

2.2.2 Kokkos

Kokkos is an open-source Performance Portability Framework in C++ [17][18]. It targets all major HPC platforms, such as CUDA, HIP, SYCL, and OpenMP, with several other backends in development.

Kokkos programming model has 3 main abstractions, allowing it to be agnostic and performant:

1. Computational kernels (such as `parallel_for`) are executed within an *execution space*.
2. These kernels operate on data residing in *memory spaces*.
3. Data is in multidimensional arrays with *a polymorphic data layout*.

The first two abstractions are to allow execution on the so called *manycore* architectures (i.e., GPUs and multi-core CPUs), where the execution space is an abstraction of any specific hardware device on which we execute, and the memory space of memory either on the hardware device or the system itself (more in subsections 2.2.2.2 and 2.2.2.3).

2.2.2.1 Memory Layouts & Views

Arguably the most interesting feature of Kokkos relates to the third abstraction. Having a polymorphic data layout means that, based on the target architecture, Kokkos can choose a specific way to store data so that the memory access pattern is ideal for that specific device architecture. For instance, CPUs benefit from a blocked data access pattern, while GPUs benefit from a coalesced data access pattern (see figure 2.2). Depending on which architecture we are targeting, an identical code snippet defining some data will utilize a different layout in memory. This is crucial, as memory access patterns are very important when it comes to performance.

Two main memory layouts are

- `Kokkos::LayoutRight` – row-major, default for CPU
- `Kokkos::LayoutLeft` – column-major, default for GPU

If these are not specifically set via template arguments, then the default is assumed based on the target backend.

Kokkos uses its own container supporting memory layouts, that being `Kokkos::View`. It is a pointer to a potentially multi-dimensional (at most eight) array for storing data. While the type of data and dimension count must be stated at compile time, dimension sizes can be set at runtime. Views behave like `std::shared_ptr` in that they use reference counting for deallocation. When declaring a `Kokkos::View`, a plethora of template arguments can be specified, and those dictate the attributes of the `View` in question.

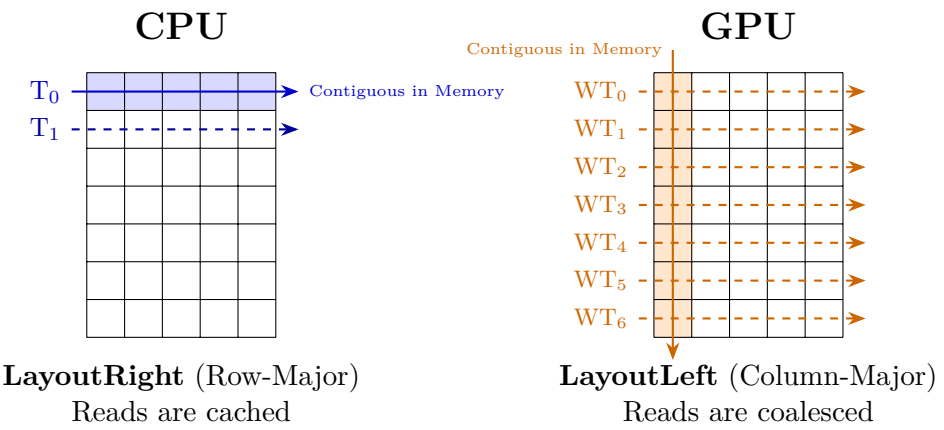


Figure 2.2 Memory layouts paired with ideal architectures. For **CPU + LayoutRight**, each core can read the whole cache line. For **GPU + LayoutLeft**, all warp threads (WT_n) can read their part of cache line at once.

```
template <  class DataType
          [, class LayoutType    ]
          [, class MemorySpace   ]
          [, class MemoryTraits  ]
          >
class View;
```

As seen in the snippet above, `View` takes four template arguments. `DataType` is the type to store and is mandatory, while the remaining three are optional. `LayoutType` serves to specify previously mentioned memory layout. When we want to explicitly state where the `View` will reside, we can set `MemorySpace` (see subsection 2.2.2.3). Lastly, we can provide further hints for the compiler regarding the expected use of our `View` via the `MemoryTraits` argument. For instance, if we know the data will be accessed irregularly (e.g., non-sequentially) and read-only, we can tag the `View` as `RandomAccess`. This allows hardware specific optimizations when available, such as when targeting CUDA, at no cost for other targets (no-op) [19].

2.2.2.2 Execution Spaces

As alluded to in subsection 2.2.2, **Execution Space** is the fundamental abstraction that represents **where** (as in on what hardware) and **how** (as in which variant for that hardware is used) execution is done (see figure 2.3). There are many specifically defined execution spaces, such as:

- `Kokkos::Cuda`
- `Kokkos::HIP`

- `Kokkos::SYCL`
- `Kokkos::OpenMP`
- `Kokkos::Threads`
- `Kokkos::Serial`

But trying to account for them like this defeats the whole purpose of PPFs. Thats why we simply don't. Instead, we utilize aliases that change based on compilation settings (i.e., target backend):

- `Kokkos::DefaultExecutionSpace`
- `Kokkos::DefaultHostExecutionSpace`

There are two default aliases because the assumption is that our hardware configuration is made up of the main system (i.e., Host) and accelerators or coprocessors such as GPUs (i.e., Device). Based on our target, these aliases are set to the corresponding architecture.

For instance, if we were to target a machine with a **single core CPU** unable to run parallel code (however strange this setup choice may be) and an **AMD GPU accelerator**, these aliases would then be:

- `Kokkos::DefaultExecutionSpace = Kokkos::HIP`
- `Kokkos::DefaultHostExecutionSpace = Kokkos::Serial`

When we launch a kernel, such as `parallel_reduce` (more in subsection 2.2.2.5), we can then use these aliases to specify whether we want the code to be run on the Host or Device (if available) by passing one or the other. By default, Kokkos looks at `DefaultExecutionSpace`. If no Device is available, this will match `DefaultHostExecutionSpace`. This is the core mechanism allowing us to write logic completely detached from the hardware on which it will be executed, yet still be able to run on all supported backends.

In addition to this, architecture specific hints can be provided when compiling to further optimize for a given target. Suppose we know that our target machine has an **NVIDIA A100 GPU** and an **AMD Epyc CPU** with **Zen 2** architecture. Then we can provide the corresponding compile flags — in this case `Kokkos_ARCH_ZEN2` and `Kokkos_ARCH_AMPERE80`.

2.2.2.3 Memory Spaces

Memory Spaces function tightly coupled with execution spaces as an abstraction for the physical location of data within the complex memory hierarchy of the target machine. Because heterogeneous systems have multiple areas where data can be stored (see figure 2.4), with varying ease of access and speed of communication between them, it is important to specify where we want the

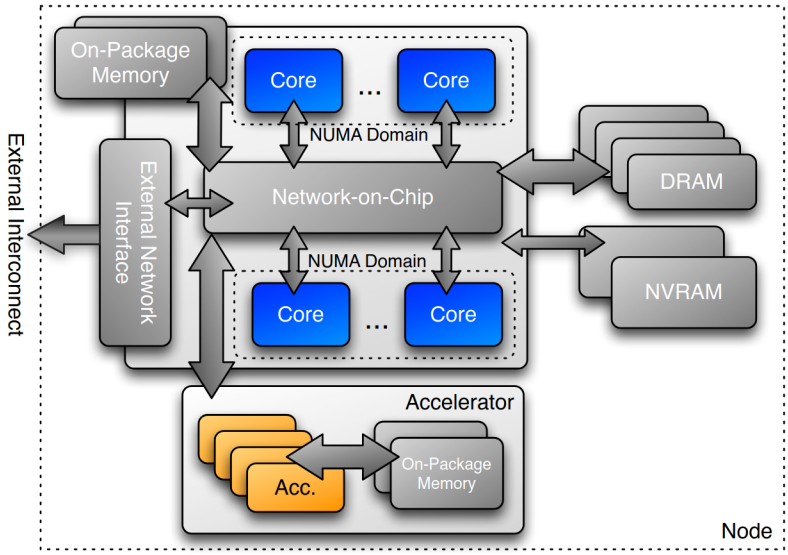


Figure 2.3 Diagram of heterogenous system with various places to execute code colored [20].

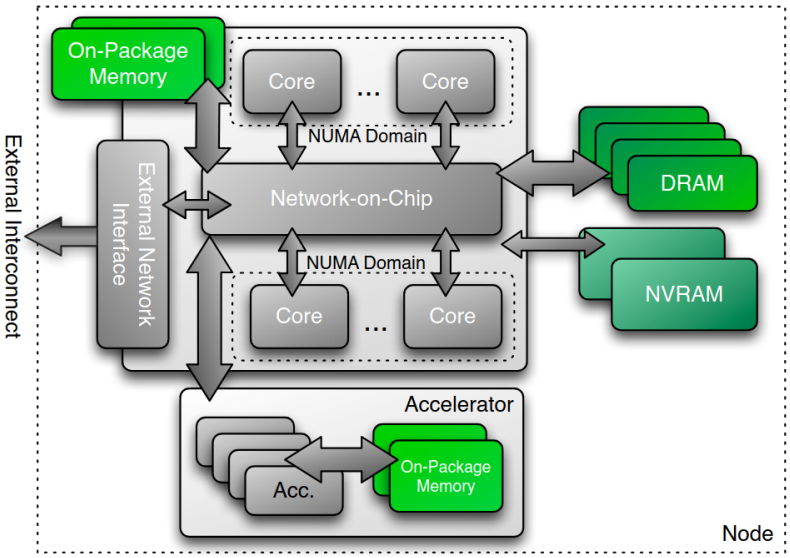


Figure 2.4 Diagram of heterogenous system with various places to store data colored [20].

data to be stored. This can, for instance, make the data inaccessible from execution spaces other than the Device whose memory space we are using.

Akin to execution spaces, there are many specific memory spaces, such as:

- `Kokkos::CudaSpace`
- `Kokkos::HIPSpace`
- `Kokkos::SYCLDeviceUSMSpace`
- `Kokkos::HostSpace`
- `Kokkos::SharedSpace`

And like with execution spaces, we don't handle them directly. Instead, we ask the execution space in which we plan to execute for its memory space — for instance, like this:

```
1 // Accelerator memory space (CudaSpace, HIPSpace, ...)
2 Kokkos::DefaultExecutionSpace::memory_space
3 // System memory space (HostSpace -- RAM)
4 Kokkos::DefaultHostExecutionSpace::memory_space
```

Note that there is an exhaustive list of additional variations of memory spaces for many targets, but it is discouraged to use them directly. We should ask for memory space tied to our desired execution space, whatever that space may be (CUDA, HIP, Host...). Suppose we are trying to initialize a `Kokkos::View` in memory space `Kokkos::HIPSpace`, while our machine has an NVIDIA GPU — this clearly breaks our device-agnostic codebase and thus necessitates the usage of execution spaces as aliases from which we can request memory space.

2.2.2.4 Data Movement

With the introduction of memory and execution spaces, a natural question comes to mind: how can we manage and move data across memory spaces, for instance, to offload computation onto the Device where we want to execute it? There must be a way because any application, by necessity, starts in the Host execution and memory space, as that is the Kokkos model description of the CPU executing our program. We cannot simply execute the kernel on the GPU, telling it to process data present in the CPU memory space (i.e., RAM of our machine). We need to make sure the data are accessible from the execution space of the Device on which we want to process them. This means having them in the GPU VRAM if we want to process them on the GPU.

Kokkos defines and enforces a set of accessibility rules via the template trait class `Kokkos::SpaceAccessibility`, evaluated at compile time. It takes

an execution space and a memory space as template arguments and evaluates whether memory allocated in the specified memory space can be **safely** and **efficiently** accessed by a thread executing within the specified execution space. Kokkos checks if direct access is either physically **impossible** (e.g., a CPU thread attempting to dereference a pointer mapped to GPU memory) or **extremely slow** (e.g., a GPU reading Host memory over the PCIe bus) — in which case, it necessitates explicit data migration.

For data migration, A view in the target memory space is created, and data is copied over by explicitly calling `Kokkos::deep_copy`. Kokkos also offers **Mirror View** concept for a more sophisticated means of migrating data and keeping both Host and Device data synced, but for one-off transfers to the Device, explicitly calling for a deep copy is sufficient.

2.2.2.5 Parallel Dispatch

While previous sections tried to describe the principles of the Kokkos programming model, this section will explain the *user layer* — how we can execute actual computational work within the Kokkos model.

Suppose we have an algorithm we want to write using Kokkos so that parts of it can be executed in parallel on any of the supported backends. There are three parts to this — we must be able to **encapsulate the algorithmic logic** and fit it into a **Parallel Pattern** and **Execution Policy** offered by Kokkos. To achieve the first, we pass our logic either by functors or lambda functions [20].

As for the parallel execution patterns, there are three main ones:

- `parallel_for`: The most ubiquitous pattern, designed to execute a function over a defined iteration space an arbitrary number of times. Crucially, it executes in an undetermined order, implicitly asserting that the loop iterations are entirely independent and free of data races. This is a very common and useful pattern for parallel code. For instance, it is ideal for efficiently setting a vector mask.

- `parallel_reduce`: Combines `parallel_for` with a thread-safe global reduction operation. This pattern safely aggregates results from threads to compute global metrics, such as work contribution per iteration and similar.

- `parallel_scan`: Executes a parallel prefix or postfix scan across an iteration space. While less common than standard loops or reductions, it is vital for algorithms requiring cumulative sums or dynamic array compaction — case in point being our own graph construction code (see code snippet B.3).

In tandem with these patterns, we also specify **the Execution Policy**, which dictates specifics regarding the dimensions, concurrency, and grouping

of the threads dispatched to execute them. There are two main policy groups, those being **Range** and **Team** policies.

- **Range Policies**: used to execute an operation once for each element in a range. There are no prescriptions regarding the order of execution or concurrency, which means that it is not legal to synchronize different iterations.
- **Team Policies**: models hierarchical parallelism by clustering parallel execution threads into isolated groups called *teams*. When launching a kernel with a team policy, the developer specifies two primary dimensions:
  - *league size*: the total number of independent teams, analogous to the number of thread blocks in a CUDA grid [13].
  - *team size*: the number of concurrent threads within a single team, analogous to the block size [13].

With these three things selected, suppose we want to launch a kernel to set a boolean mask. We would then write something like:

```
// "boolean mask for N elements"
Kokkos::View<bool*> active_mask("ActiveMask", N);

// Pattern: parallel_for
// Policy: Implicit RangePolicy 0 to N (i = 0..N-1)
// Encapsulation: KOKKOS_LAMBDA
Kokkos::parallel_for("MyLabelForDebug",
                     N, KOKKOS_LAMBDA(const int i)
{
    active_mask(i) = false; // unset each element
});
```

In this snippet, Kokkos automatically maps the iteration space (from 0 to $N - 1$) across the available threads of the underlying execution space. The `KOKKOS_LAMBDA` macro ensures the compiler properly captures the loop body and makes it callable on the target Device (i.e., *slots in* the correct directives).

There are cases, however, where a simple range policy might lead to severe thread underutilization. A fitting example in the context of this thesis is processing edges of vertices in a graph. This inherently leads to highly irregular workloads because, while the majority of threads in a warp can process low degree vertices quickly, they might have to idle and wait for a thread working on a comparatively extremely high degree vertex. To combat this, we can leverage the previously mentioned **Team Policies** for hierarchical parallelism. Instead of assigning one thread per element, we can assign an entire *team* of threads to a single vertex. This allows the threads within that team to cooperatively process the vertex's incident edges:

```
// TeamPolicy:
//  league_size = N (vertices),
//  team_size = AUTO
using team_policy = Kokkos::TeamPolicy<>;
using team_member = team_policy::member_type;

Kokkos::parallel_for("ProcessVertexEdges",
    team_policy(N, Kokkos::AUTO),
    KOKKOS_LAMBDA(const team_member& team)
    {
        // The league rank = vertex this team is handling
        int u = team.league_rank();

        // Threads within this team can iterate over
        // u's edges using nested patterns like TeamThreadRange
        // ...
});
```

By letting Kokkos choose the `team_size` via `Kokkos::AUTO`, the framework optimizes the block dimensions for the specific hardware backend in hopes of preserving performance portability.

Chapter 3

Current landscape & Reference Implementations

3.1 Current landscape

As of today, the landscape of exact maximum flow solving is potentially undergoing a paradigm shift — transitioning from the long-established Push-Relabel-based approaches to a recent theoretical breakthrough in *Almost-Linear Time* algorithms laid out in [21][6]. It is important to realize that this is still mainly a theoretical frontier.

In practice, the best approach is often dictated by how specific you can be regarding the expected input for your use case. This makes sense, as some problems commonly treated as max flow or min cut show a very predictable and uniform topology pattern of their inputs. A textbook example of this is image segmentation and computer vision, where the Boykov-Kolmogorov algorithm [4] is widely regarded as the best approach. On the other hand, it lacks a strongly polynomial time bound, which makes it vulnerable to specific worst-case graph topologies.

When looking for a generic solver, where we assume nothing about the input topology, push-relabel and its variants seem to remain among the most dominant and widely used approaches. While the local nature of its operations invites parallelization at first glance, actually mapping push relabel onto parallel ready hardware is rather challenging. This is mainly due to the shared memory state of the graph and the workload imbalance caused by differences in the degrees of the vertices.

To contextualize the performance and correctness of our proposed implementation (see chapter 4) within this landscape, we selected four reference solvers representing different general solver approaches, targeting both sequential and parallel execution on both CPU and GPU. They are outlined in the following section.

3.2 Reference Implementations

In this section, we briefly introduce and outline the reference solvers used in this thesis. We also include additional notes and details regarding any irregular behavior encountered in the Experimental Evaluation chapter (see chapter 5).

3.2.1 hpf

Hochbaums pseudo flow (hpf) algorithm is unique in that it offers a different conceptual approach compared to traditional augmenting path or Push-Relabel algorithms. It first solves the **maximum blocking cut problem**, and from it recovers the flow with a complexity of $O(mn \log n)$ on a graph with n nodes and m edges. It not only adds the notion of **excess**, like in Push-Relabel algorithms, but also **deficit**, allowing the violation of flow balance in both directions [22][23].

Implementation was sourced from [24] and the **pseudo_fifo** variant was used, as the page states it is the fastest one in general. It takes input in the standard DIMACS format.

3.2.2 hipr4

This sequential implementation uses **hi**ghest-label **P**ush-**R**elabel approach. **hipr4** uses this selection rule to determine which vertex to discharge at any point [25].

Implementation was sourced from **the 13th DIMACS challenge** page at [26]. There, it is listed as one of the reference solvers. It takes input in the standard DIMACS format. This solver is usually the best one on the DIMACS challenge data sets. We did modify this implementation very slightly, only by increasing the precision of the timer output to four decimal places to match the rest of our solvers.

3.2.3 ECL-Maxflow

This is GPU native solver utilizing CUDA [27]. It is based on standard Push-Relabel algorithm, but with added optimizations and improvements. For example, it utilizes a two-level parallelization scheme, where it employs an entire warp (32 threads) per element to improve load balancing. It is said to be an improvement over the state-of-the-art and is written in CUDA.

Implementation was sourced from [28]. It takes input in its own **ECL binary format**, which is not an issue since the implementation includes a program to convert the standard DIMACS format to it. The conversion of data was not accounted for in the benchmarking process.

3.2.3.1 Notes

In our environment, this solver has been inconsistent regarding correctness with some inputs. Specifically, three patterns emerged:

- **Non-deterministic results:** Out of the ten runs per input, it returned multiple different results, sometimes with the correct one included and sometimes not. More importantly, incorrect results were sometimes larger than the actual maximum flow (as calculated by hpf, which was our source of truth)
- **Crashing:** Rarely, ECL-Maxflow would crash, potentially causing the GPU to become unreachable, as if it were disconnected. Rebuilding the devcontainer we used made it appear again.
- **Errors:** On some inputs, the solver errored out, stating that it did not return all flow to the source, followed by source and sink excess.

For a more detailed description of the behavior, see subsection 5.3.1. Because it is outside the scope of this thesis and would certainly entail a lot, we did not delve much deeper into these issues or attempt to analyze them fully. It is, of course, possible that our setup or inputs were the source of instability.

For full transparency, let us describe the way we built the binary. We simply cloned the repository and followed the root readme file, as any user would. We set `DEVICE_CC = sm_80` to reflect the compute capabilities of our hardware and then executed `make`. This produced the solver binary.

3.2.4 PBBS-Syncpar

This is a parallel CPU implementation of the PRSN algorithm outlined in section 1.4.2 and first described in [11] in 2015.

Implementation was sourced from [29], which should be the official repository tied to the original whitepaper. It takes its own **PBBS binary format**, but like ECL-Maxflow, it also contains a program for conversion from the standard DIMACS format. The conversion of data was not accounted for in the benchmarking process.

3.2.4.1 Notes

In our environment, this solver has been inconsistent regarding correctness with some inputs. Specifically, two patterns appeared:

- **Non-deterministic results:** Out of the ten runs per input, it returned multiple different results, sometimes with the correct one included and sometimes not. As far as we can tell, it never returned a result **larger than** the actual maximum flow, leading us to speculate that this could be a race condition triggering early convergence.

- **Deterministic wrong results:** On some inputs, all ten runs return the same supposed max flow, but it is incorrect.

For a more detailed description of the behavior, see subsection 5.3.1. Same as with ECL-Maxflow, we did not conduct an extensive analysis of the implementation. It might very well be that it relies on certain CPU architecture details, ways of handling multi-threaded code, or that the inputs trigger some bugs or edge cases.

Let us also describe the way we built the solver for potential reproducibility and transparency, as we did with ECL. Since the repository contains makefiles and some documentation within readme files, we simply cloned the repository and navigated to the solver directory at `./benchmarks/maxFlow/syncPar/`, where we executed `make`, as a standard user would. This produced the solver binary.

Chapter 4

Design & Implementation

This chapter describes our device agnostic max flow solver implemented in Kokkos. It is nicknamed **KNFS** (Kokkos Network Flow Solver), and the design of its entire pipeline is described in the following sections. In addition to that, we outline an alternative approach utilizing Kokkos team policy to perform hierarchical parallelism (see section 4.7).

While the following sections try to describe the design of the solver, the actual implementation code can be found at [30]. Look at the **main** branch for the primary KNFS solver and the **dev_multiparallelism** branch for the alternative approach.

4.1 Overview

Our Implementation has multiple parts that create a complete pipeline for loading a graph from the standard DIMACS format, converting it to our internal representation, initializing it on the Device where we will execute computation, and then running the actual solver on it. It can be split into three phases, those being **loading**, **building**, and **solving**. A top down view of this pipeline is visualized in figure 4.1. Each individual part of the solver is further explained in their respective section.

The first part of our pipeline deals with loading and parsing the graph from a file into a view of edges in Host memory. This view is then moved to the Device, where we build it into a valid graph representation. The last step is to do the initial push of flow from the source vertex. After this concludes, we have a fully built and correctly initialized graph on the target Device. At this point, the solving phase and the actual solver logic itself start.

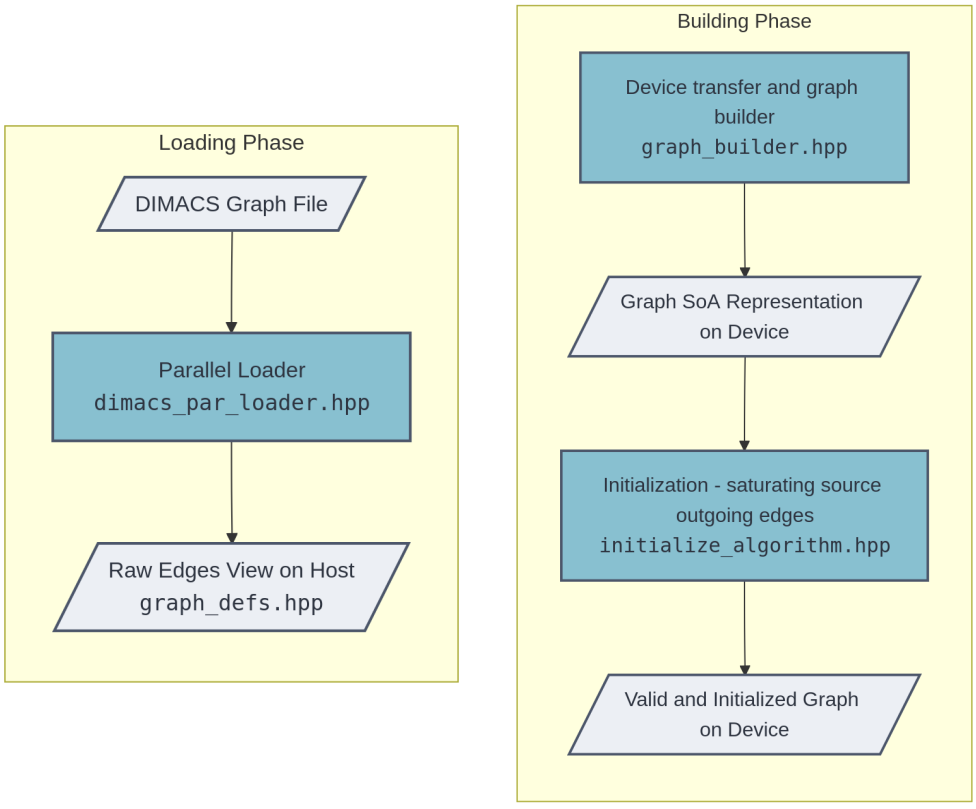
The KNFS approach is based on the PRSN algorithm, as it conceptually performs the same four steps described in subsection 1.4.1 and utilizes a deterministic lock-free criterion described in subsection 1.4.2. These four steps are merged into two kernels, called **process** and **apply** kernels (see subsections

4.5.1 and 4.5.2). The process kernel performs steps $1-2$, alternating between them as long as there are valid push paths and excess to push. The apply kernel performs steps $3-4$ to commit to local buffered changes of the process kernel. We consider these two kernel launches, modulo the occasional global relabel (see section 4.6), as a single iteration of the solver.

We repeat this as long as there are active vertices at the beginning of the current iteration, treating it as our convergence condition. To track active vertices, we use a two-queue buffering system (see section 4.2) with the *current* and *next* queues. In each iteration, the process kernel is launched over vertices from the *current* queue while enqueuing new ones into the *next* queue. Between iterations, we swap the queues: the *next* queue becomes *current*, and the old queue's size is reset to zero, allowing its memory to be overwritten by the next batch of active vertices.

We practically always start the solver phase with active vertices, as the initialization step of the building phase saturates edges from the source vertex, thus activating its neighbors. In cases where the source has a degree of zero or is only adjacent to the sink vertex, we do not enter the solver loop at all, immediately returning the maximum flow pushed during the initialization.

This concludes the overview of our solver and the interplay between its parts. As was already mentioned, detailed description of any single part can be found in the following sections.



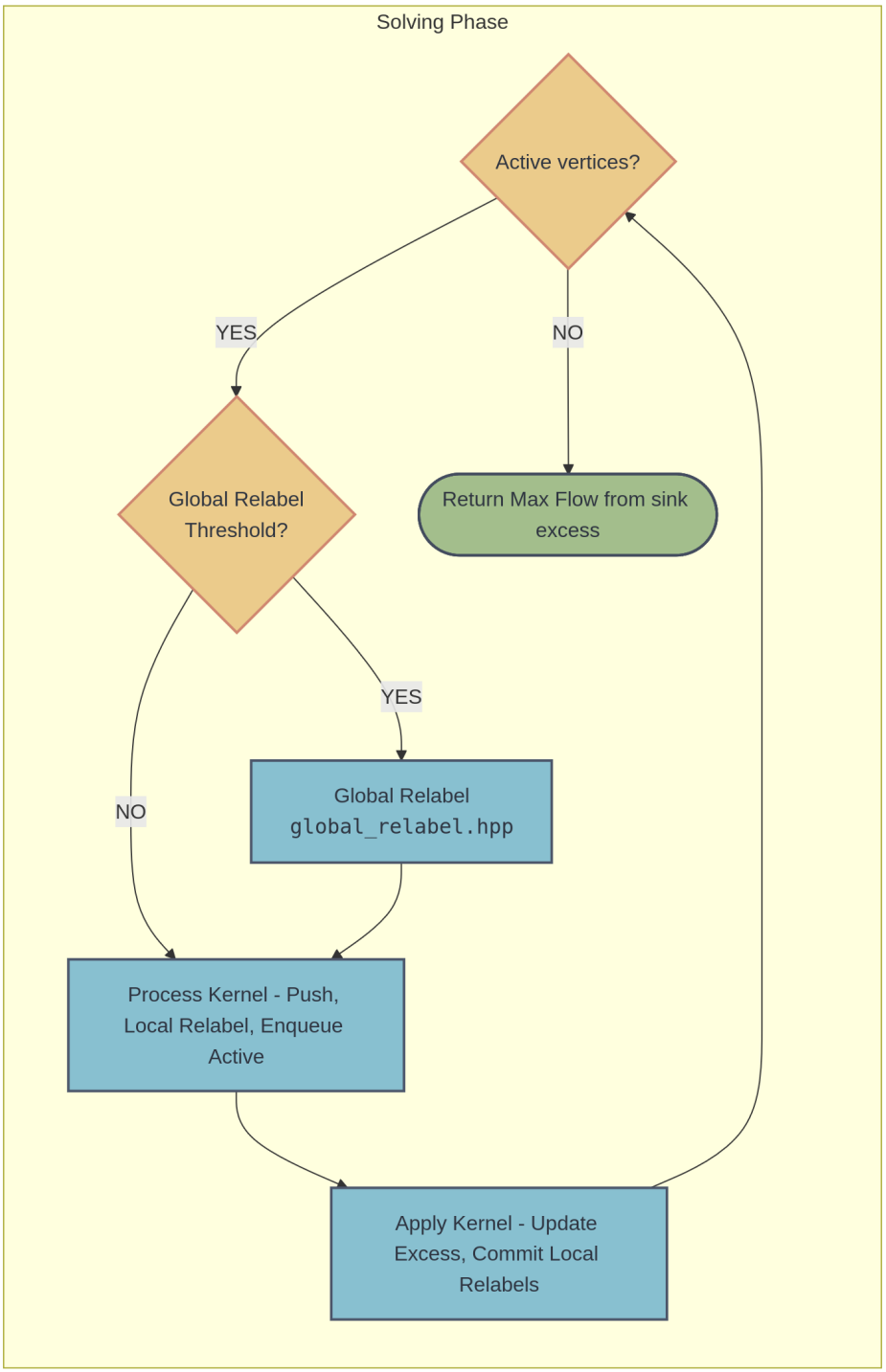


Figure 4.1 Flowchart of **Loading** → **Building** → **Solving** phases of our solver. Logic flows top to bottom [31].

4.2 Graph representation

In our implementation, Graph is represented via the `Graph<DeviceType>` templated structure, which adheres to the **Structure of Arrays** design pattern. This is templated so that all the internal arrays (views) can deduce their execution and memory space and optimal memory layout. Within it, the actual flow network is represented using a Compressed Sparse Row (CSR) format. It can be found in `graph.hpp`

Views of the representation can be divided into four categories:

- **Topology Views:** The CSR structure for describing the relations between Vertices and Edges. It is made up of two views — `row_map` (size $|V|+1$) and `entries` (size $|E|$). Both remain strictly read-only during the execution of the algorithm. The CSR format was chosen as it ensures continuous reading of the edges of any given vertex, which is a crucial operation in our algorithm.

- **Edge Properties:** Views for `residual_capacity` and `reverse_edge` index (size $|E|$). `reverse_edge` is read-only, while `residual_capacity` is intuitively both read and modified during the algorithm.

- **Vertex Properties:** Algorithm state relies on views for `excess` and `label` (i.e., distance d). To prevent race conditions and minimize atomics, changes to these are buffered to `added_excess` and `new_label`. This follows PRSN and is crucial, as buffering labels allows for local relabeling within the pushing phase, thus making more pushes per iteration possible. Buffering excess requires atomics because many vertices can try to push to a specific vertex at once.

- **State and Work Queues:** Active vertices are tracked using dense, dynamically sized queues (`current_active` and `next_active`). To deduplicate queue insertion without the overhead of $O(|V|)$ bitmask clearing between every iteration, we use `active_iteration_mask` (size $|V|$). This view stores the iteration number k when the vertex was last enqueued. To insert a vertex into the next queue, a thread performs an atomic exchange with $k+1$. If the previous value was not $k+1$, the thread successfully claims the slot and enqueues the vertex. This approach eliminates the need for a global clearing pass between iterations.

  We also have `active_phase` similar to `active_iteration_mask`. This mask is used during the **process kernel** (see subsection 4.5.1) to safely and quickly verify whether the vertex entered the kernel as active to determine if we need to check the winning criterion (see subsection 1.4.2) or not. While we could query the iteration mask, it is not safe as we modify it during the kernel.

Accounting for all of these views, our graph representation can be defined roughly as shown in appendix B.1.

4.3 Graph Loader

To load graphs from disk, we implemented an optimized parallel loader (in `dimacs_par_loader.hpp`). We naturally utilized Kokkos to implement a two-pass parsing strategy executed on the Host. As for the input, it accepts the standard DIMACS format file.

Rather than relying on parsing via file streams, our loader maps the entire input file directly into the Host's virtual address space using memory mapping (`mmap`). This avoids redundant data copying and standard I/O system call overhead. Once mapped, the loading process proceeds in three stages:

- **Header scan:** A sequential scan of the file's initial bytes extracts global metadata, namely the total number of vertices and the exact identifiers for the source and sink nodes.
- **Chunking & counting pass:** Mapped memory is divided into chunks among available threads. Crucially, these chunks are aligned to correspond with the nearest newline, as we might otherwise split the chunk mid-line. `Kokkos::parallel_for` is then used to count edges in each chunk. This determines the exact starting index (offset) in the final view where each thread will begin writing.
- **Parse & fill pass:** A second `Kokkos::parallel_for` is launched, now with each thread knowing their offset, so all can write to the final view `HostEdgeList` in parallel.

Once this construction is complete, the `HostEdgeList` is passed to the Graph Builder. For the parsing of data, we use fast and generally *low level* functions, such as `std::from_chars` and `std::memchr` (for SIMD instructions). This prioritizes speed above general code safety (as in locale checks and format validation), as we know the expected valid input format, and these functions are called many times during the graph loading.

Since the only requirement for a loader to work in our pipeline is that it stores edges in the expected view (defined in `graph_defs.hpp`), which is then passed along, a different or format-specific loader could be swapped in somewhat easily. That being said, our loader has a throughput of over six gigabytes per second, making it practically I/O bound on our benchmark machine.

Graph loader code snippet can be viewed in appendix B.2.

4.4 Graph Builder & Init

After our loader collects all the graph information into a kokkos view on the Host, it is passed to our graph builder (in `graph_builder.hpp`). Its objective is to handle the transition from an unstructured list of edges on the Host to our specified graph representation in the memory space of our target execution Device. This is done via a common parallel data-compaction and indexing approach adapted to utilize kokkos parallel primitives, further described below.

4.4.1 Graph Builder

The first thing we do in our implementation is move the edges received from the graph loader onto the Device, so we can do the majority of the work in the target memory space, preferably in parallel. This way, the final graph will be ready and assembled where we want it for the algorithm. The building process then consists of the following phases:

- **Allocation & Moving edges to Device:** The builder first allocates Device memory sized at twice the input edge count to accommodate the necessary reverse edges. Edges are then bulk-copied from the Host to the Device.
- **Adding Reverse Edges:** To create a correct residual network, every directed edge needs a corresponding reverse edge to track residual capacity. We launch a parallel kernel over the first half of the allocated edge view (that is, over the original edges), generating a reverse edge for each (with capacity 0) in the second half of the edge view.
- **Sorting:** This edge view is then sorted with `Kokkos::sort`. This relies on overloaded comparisons (in `graph_defs.hpp`) such that the source node id takes priority over the destination.
- **Deduplication & Compaction:** We then deduplicate and merge edges between vertex pairs into one. This is done in a parallel for loop, marking the first occurrence of each unique edge with a flag. A subsequent inclusive parallel scan over these flags calculates the precise memory offsets (write indices) for the CSR representation. This mutates the graph in case it has multiple edges between a vertex pair, though we sum up the capacities atomically in the next step, so the resulting graph is still equivalent.
- **CSR Fill:** Edges are written into the final view, representing the graph topology. This is done via a parallel for loop. For deduplicated multi-edges, only the first thread of a duplicate group writes the graph topology, while all threads atomically add their respective capacities.

- **Row Map Fill:** Since the final edges are written, we can now calculate the degrees of each vertex so that we know which index in the edge view we should point to for a given vertex. This is done in a parallel for loop, which computes the degree of each vertex using atomic increments. Then a parallel scan performs a prefix sum to convert these degrees into the final offsets in the edge view.
- **Reverse Edge Linking:** To allow $O(1)$ lookup during the algorithm, each edge must have its reverse linked. Given that the edges of the vertex are sorted and placed in sequence in memory, we can link them in parallel by having each thread corresponding to an edge run a binary search on the destination vertex edges, linking the reverse once found.

  Graph builder code snippet can be viewed in appendix B.3.

4.4.2 Initialization

After graph building concludes, we are left with our valid representation of the flow network. However, before we can kick off the algorithm, we must initialize the graph by saturating the edges going out of the source node.

This is done in a single `Kokkos::parallel_reduce` over the outgoing edges of the source. In this kernel, each thread simply pushes along its edge to the neighboring vertex, updating its excess. Since our builder deduplicates multi-edges, this can be done without atomics. After this, each thread enqueues their neighbor vertex via `Kokkos::atomic_fetch_add`, so that they start active. The reason this is done in the reduce kernel is so that we can accumulate the flow pushed from the source. We also need to set the source height label to the number of vertices in our graph. Initialization can be found in `initialize_algorithm.hpp`. Its code snippet can be viewed in appendix B.4.

4.5 Main kernels

The core computational part of our implementation is captured within *process* and *apply* kernels, which reflect steps $1-2$ and $3-4$ of the **PRSN** algorithm (see subsection 1.4.1), respectively. They are in `main.cpp`, where an overview of the entire algorithm orchestration and runtime logic is present (see section 4.1).

4.5.1 Process Kernel

The process kernel is a parallel for loop over active vertices. Each thread receives its vertex u from the queue of active vertices. As long as u has flow to push, we iterate over its outgoing edges to see if we can find any admissible edges to push along. An admissible edge $u \to v$ is an edge with positive capacity and a label 1 below that of u; this is so that we uphold the rule of *pushing downhill*, which is necessary for the push-relabel correctness (see subsection 1.2.3). We utilize the deterministic win criterion used in PRSN (see subsection 1.4.2) so that we can push flow along the edge without atomics, provided we won. To determine if v is active, we utilize `active_phase` view to check if it was active at the beginning of this process kernel iteration. The excess of v must be updated atomically because it might be receiving flow along some other edge $w \to v$. If we successfully pushed flow to v, we know for a fact it will have excess in the next iteration, and thus we enqueue it. Provided we still have excess, we move onto to the next outgoing edge from u.

After we have processed all edges, we are either still left with excess, or we broke early from the loop the moment we depleted it. If we have leftover excess and we did **NOT** skip any admissible edge (by losing the win criterion), we relabel v to be exactly 1 above its smallest label neighbor with a positive capacity edge, so they become admissible. After this relabel, we process the edges again to try and get rid of our excess (this is the merge of steps $1-2$ of the PRSN algorithm). Note that if we skip an admissible edge, we must break from the process kernel straight away; otherwise, we could relabel above it, breaking the algorithm correctness.

Once we break out of the flow pushing loop, we write back our excess (if any) to u and enqueue it for the next iteration if u has excess or has been relabeled. Kernel code snippet can be viewed in appendix B.5.

4.5.2 Apply Kernel

The apply kernel is a parallel for loop over the next iteration's active vertices. Each thread receives its vertex u, adds to its excess the flow pushed to it by neighbors in the process kernel, and updates its label to reflect the local relabels done in the process kernel. Kernel code snippet can be viewed in appendix B.6.

4.6 Global Relabel

The global relabel step is done via parallel BFS from the sink, executed on the Device. It is launched periodically after a set amount of iterations, by default 1500. Implementation is found in `global_relabel.hpp`. Our implementation uses a two queue frontier design, utilizing the two queues `current_active` and `next_active` defined in the graph representation (see section 4.2).

We start with a singular parallel pass over **all** vertices, setting their labels appropriately. This means that the sink gets 0, the source gets N, and any other vertex is set as *unreachable* with a label of $N + 1$. We also add the sink vertex to the `current_active` queue for processing as the BFS starting point.

Then a sequential loop (at most N iterations) is executed, in which we process frontier vertices in parallel, labeling their yet unvisited (label $N + 1$) neighbors with $current\_iteration + 1$ and enqueuing them to the other queue for the next iteration if they have any residual capacity (meaning flow can be pushed through them to sink). Queues then swap each iteration, so that we always read the current active vertices from one queue and write the next iteration's active vertices to the other (same approach as the solver active queues swap logic). This is done until we have no more active vertices or hit N^{th} iteration, because at that point we must have visited all reachable vertices no matter what. For the worst case scenario, our graph would have to be a chain of vertices with the source at one end and the sink at the other.

The global relabel code snippet can be viewed in appendix B.7.

4.7 Degree-based modification

In addition to the main implementation described in the previous sections, a proof-of-concept for an **alternative approach**, which attempts to **utilize hierarchical parallelism** through the use of Kokkos team policies, was created. This alternative implementation is fully functional and correct, though naively implemented and not fully developed, as it fell outside of the main scope of our work. Due to this, performance was overall worse than our primary solver. For brevity, we won't be including its code snippets within the thesis, though they will be available in the repository. It still goes to show the ability to use even more advanced kokkos features to unlock more complex and specialized approaches.

The idea behind this implementation is to try to solve the potential major bottleneck of the previously described approach; significant workload difference between vertices. Suppose you have a graph modeling the linking of websites or representing densely packed clusters connected sparsely, akin to representing a map of roads in and between cities. If our input has large differences in the degrees of vertices, it can lead to a situation where many threads quickly process low degree vertices and are then left waiting for one thread that has to process a cluster vertex with a comparatively massive degree. Scanning over outgoing edges is done sequentially and thus can become a major bottleneck, leaving the whole warp idle.

Even if the graph is balanced, having no major differences in degrees from the average degree of the graph, there is still performance left on the table by forcing sequential processing of the edges, even if we are working in parallel over vertices. To tackle this, we decided to adapt a hybrid approach based on the vertex degree.

Instead of launching one processing kernel over all active vertices, we introduce two separate processing kernels for low and high degree vertices. Low-degree vertices are those with a number of outgoing edges being $\leq$ the warp (or its equivalent) size on our target backend. Their kernel is very similar to the one described in the previous subsection 4.5.1.

As for the high-degree vertices, they are processed with team policy based parallelism over both the vertices and their edges. A high-degree vertex is processed by a team of threads sized equal to the warp size. The team then processes warp sized chunks of the edges in parallel. This means that there is still sequential processing of edges, but at a larger stride. This has some benefits — for instance, it allows the introduction of current arc (or warp, in this case) tracking to avoid rescanning warps from the beginning each time.

This shift in approach facilitated some changes in graph representation, the way we enqueue vertices, and perform global relabeling. They are outlined in the following sections.

4.7.1 Graph representation changes

Since we now recognize two types of vertices based on degree, we use two separate queues for them. This has the benefit of allowing us to launch our kernels over them easily. We can also easily query their size to know if we can skip a specific process kernel if there are no active low/high vertices in that iteration. This means we replaced the old queues (`current_active` and `next_active`) with four new queues, two for each category of vertices. We also introduce separate queues for the global relabel process, as it used the old ones before.

To accommodate the current arc (see subsection 4.7.4), we also needed to add a view sized at N elements to track them. The index of the view element corresponds to the vertex id, and the value of that element denotes which outgoing edge to resume processing from.

Of course, the addition of these new fields required modifying the graph building and initialization process to account for them and to enqueue vertices neighboring the source into their respective queues.

4.7.2 Low-degree processing kernel

The process kernel for the low-degree vertices is, in principle, the same as the one used for all vertices in our primary implementation (see subsection 4.5.1). The only two major changes are the inclusion of the current arc and the enqueuing of vertices to which we pushed flow — we must query their degree and enqueue them based on it.

4.7.3 High-degree processing kernel

The process kernel for the high-degree vertices is a heavily modified variation of the primary implementation kernel.

Instead of sequentially iterating over all outgoing edges of vertex u, we iterate over chunks of edges and process those in parallel within each chunk. Each thread from the team gets one edge from the chunk. A problem arises with the realization that we cannot simply command the threads to *push as much as the edge allows* in parallel because they all share one resource, namely the excess flow itself. Without considering this, the solver would generate flow out of thin air at every step, essentially treating every vertex as if it were a source vertex. We solve this by separating pushing into two phases: **claiming flow** and actually **pushing it**. Each thread in a team that has its edge $u \to v$ with capacity c_{uv} eligible claims flow by placing it into the local `req_flow` variable. After that, a team-wide exclusive prefix sum scan is performed on these claims, storing the result for each thread into another local variable `exclusive_prefix`. This results in each thread being aware of exactly how much excess is claimed by threads before it in the team ordering. With this

information, pushing from all threads can be done safely by pushing

$$\min(c_{uv}, \max(excess - exclusive\_prefix, 0))$$

This also reflects another major change in approach for this kernel — all threads follow through the whole process, performing identity operations (such as pushing 0 flow) if needed. This is to prevent deadlocks, as team-wide operations might require waiting for all the threads to join them.

To clearly demonstrate parallel pushing, consider the following example:

- Vertex u has an excess of $e_u = 15$. A team has a size of 4 threads T_{0-3}:
  - T_0 finds an eligible edge with $c = 10$.
  - T_1 finds an eligible edge with $c = 8$.
  - T_2 finds an eligible edge with $c = 5$.
  - T_3 finds an ineligible edge $\Rightarrow c = 0$.
- **Claim & Scan:** Each thread claims the capacity of its edge, then after the exclusive scan:
  - T_0 has its $exclusive\_prefix = 0$.
  - T_1 has its $exclusive\_prefix = 10 = 0 + 10$.
  - T_2 has its $exclusive\_prefix = 18 = 0 + 10 + 8$.
  - T_3 has its $exclusive\_prefix = 23 = 0 + 10 + 8 + 5$.
- **Push:** Each thread pushes $\min(c, \max(e_u - exclusive\_prefix, 0))$ flow:
  - T_0 pushes $min(10, max(15 - 0, 0)) = 10$ flow.
  - T_1 pushes $min(8, max(15 - 10, 0)) = 5$ flow.
  - T_2 pushes $min(5, max(15 - 18, 0)) = 0$ flow.
  - T_3 pushes $min(0, max(15 - 23, 0)) = 0$ flow.

After this major shift from the primary solver process kernel, the rest is mostly similar, albeit adapted for dynamic enqueueing and parallel execution over multiple threads. For instance, all threads that actually pushed some flow do not enqueue the receiving vertices one by one — instead, the total number of vertices to enqueue by this chunk is summed up and reserved via one atomic fetch add from the first thread of the team.

As for the current arc, in the context of high-degree vertices, it actually notes **the chunk** from which to resume iteration.

4.7.4 Current arc

We also tried to include the current arc optimization to reduce the amount of sequential processing within the kernels. Each vertex has their arc accessible via the `current_arc` view added to the graph representation. This plays a role when iterating over edges (low-degree kernel) or chunks of edges (high-degree kernel), as we do not have to start from 0 but can resume from some previously captured edge or chunk. Do note that there are some limitations:

- **Arc is anchored to the very first skipped admissible edge:** In case we contest another vertex for an edge and lose, our arc must be set to this edge because we cannot skip admissible edges. This stems from the synchronous model of our solver, where we can alternate pushing and local relabeling to allow for more pushing within one kernel launch.
- **Arc is reset upon relabeling:** Our arc must be set to 0 when we relabel. This ties to the previous point because the moment we relabel, edges become admissible, which may be before our stored arc index.

Chapter 5

Experimental Evaluation

In this chapter, we describe our benchmarking process and the setup we used to gather the results presented in the evaluation. From the results, we aim to comment on the following:

- **Correctness**
- **Runtime comparison**
- **Performance on different architecture backends**
- **Kokkos kernel utilization and hotspots of our solver using the kernel timer**

Out of our two approaches, we focused on the primary KNFS solver, mainly due to the GPU instability and the overall lack of polish of the proof-of-concept alternative approach (see subsection 6.1.2).

5.1 Environment

Most of the thesis code development and all of the benchmarking were done remotely on the **NVIDIA DGX A100** machine provided by **CTU FIT**. Work was done in a slim devcontainer with GPU passthrough and full CPU access. Closer hardware and software specifications can be found in table 5.1.

The developer container is included as part of the thesis repository. It should be easy to build it on any system with **Docker** and an NVIDIA GPU via the **Dev Container extension** in VS Code. The container also includes Ruby, as it is a developer dependency used for helper scripts and similar tasks.

If the target machine matches our development machine, building the devcontainer and compiling the binary for the CPU/GPU target requires zero changes. For the same vendor (AMD CPU and NVIDIA GPU), but a different

architecture (not Zen 2 and Ampere), it should still work, although our configuration files specifically instruct Kokkos to compile with Ampere in mind. For a different vendor, it is pointless to use the CUDA Ubuntu image, as we do in our devcontainer — it must be supplemented by another one with vendor specific software or installed post-creation within the container.

While Kokkos is great for having one universal codebase, backend specific configurations are still necessary to some extent — there is simply no way around it. These are, however, one time setup details, and the actual codebase of our solver is never touched and compiles to them fine. For our configuration files, see the repository.

Table 5.1 Hardware and Software Specification

Component	Specification
CPU	AMD EPYC 7742 2.25GHz 64-core (Zen 2)
System Memory	512 GB DDR4
GPU	1× NVIDIA A100-SXM4 40 GB VRAM (Ampere)
Host Environment	6.8.0-94-generic, NVIDIA Driver 560.35.05
Container Environment	Docker (`nvidia/cuda:12.3.1-devel-ubuntu22.04`)
CUDA Stack	Driver API: 12.6, Runtime API (`nvcc`): 12.3.107
Compiler	GCC 11.4.0 (via `build-essential`)
Software Frameworks	Kokkos 4.5.01, OpenMP (`libomp-dev`)
Build Configuration	CMake 3.20+, Release Build (`-O3`), `nvcc_wrapper`
Target Architectures	`Kokkos_ARCH_NATIVE`, `Kokkos_ARCH_AMPERE80`

5.2 Data & Benchmarking

For all our benchmark runs, we compiled a collection of 108 input graphs representing various use cases and different topologies. Conveniently, the latest (13th) DIMACS Challenge at the time of writing is themed around network flows, so we used the datasets provided by it as a basis [26]. In addition to that, we added some street network graphs from the 10th DIMACS Challenge. This gave us our initial corpus with the following types of graphs:

- **Punch:** Road network partitioning.
- **DIMACS (1st):** Synthetic graphs from the DIMACS maximum flow challenge.
- **Vision:** Computer vision data. Groups instances by type, such as multi-view, sound, or mesh segmentation instances.
- **3Dseg:** 3D segmentation instances, originally from **the University of Waterloo** suite [32]. Includes medical image segmentation data.
- **StreetNet (10th DIMACS):** Street network data.

For each (sub)group of input instances, we sampled ten (if possible) graphs based on size to cover a variety of sizes in each group. This led to the final 108 graphs we used for our benchmark. For a list of these graphs, see table A.1.

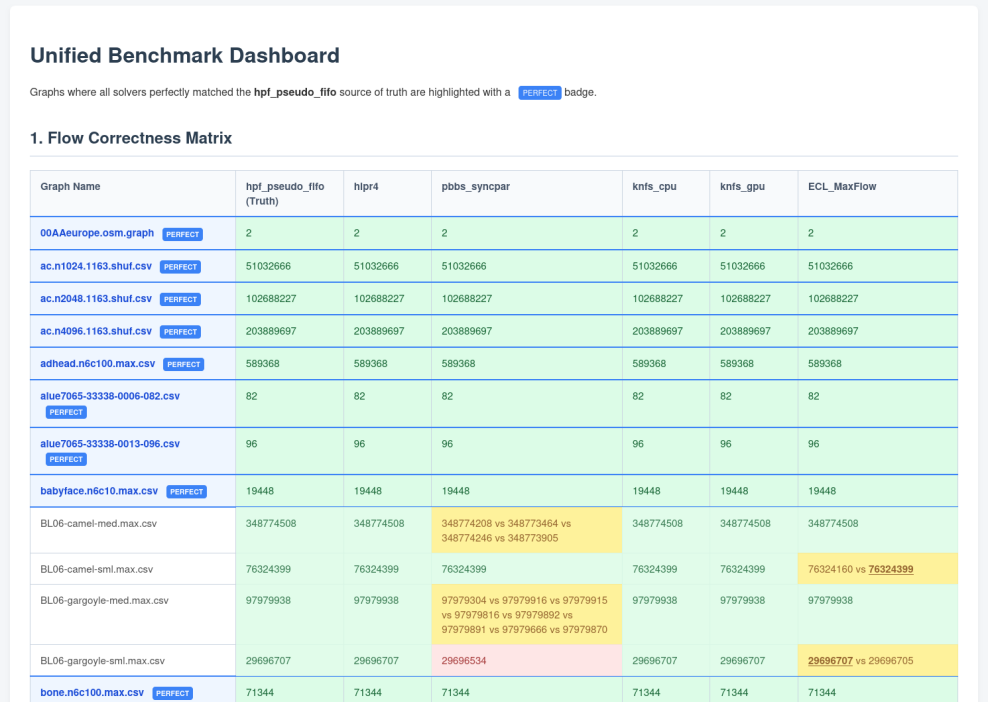
Benchmarks were run on the hardware setup described in section 5.1. Each solver was executed ten times on each graph instance (alternating between the solvers after each run) for a total of 1080 runs per solver. Each run was timed out after 300 seconds, counting the run as DNF. To automate the benchmarking process, we created a shell script and dumped the output into a `.log` file, which we then processed further for results.

This benchmarking process was run once without any profiling tools hooked in for runtime results and correctness evaluation. Then it was run again only on our solvers (KNFS CPU and GPU variant) with the Kokkos kernel timer hooked in.

5.3 Results

In this section, we present our findings from the benchmark runs. We also created a Ruby utility script for processing benchmark log files and creating a rather neat HTML summary page that presents the main metrics, namely correctness, runtime, and relative performance of solvers on each graph instance. This can be used alongside the result tables A.1 and A.2, as it provides a more visually striking presentation of them (see figure 5.1). For the final results, this report page is included in the repository alongside any other scripts.

We would also like to note that we wanted to present a master table of sorts, merging all the columns of all the tables in the upcoming sections, but we decided against it because it would be extremely wide and chaotic. Due to this, there is some forced redundancy in the tables, for instance, re-listing all the graph names. For a better reading experience, we also moved very long tables (and code blocks, for that matter) to the appendix.



Unified Benchmark Dashboard

Graphs where all solvers perfectly matched the **hpf_pseudo_fifo** source of truth are highlighted with a PERFECT badge.

1. Flow Correctness Matrix

Graph Name	hpf_pseudo_fifo (Truth)	hlpr4	pbbs_syncpar	knfs_cpu	knfs_gpu	ECL_MaxFlow
00AAeurope.osm.graph PERFECT	2	2	2	2	2	2
ac.n1024.1163.shuf.csv PERFECT	51032666	51032666	51032666	51032666	51032666	51032666
ac.n2048.1163.shuf.csv PERFECT	102688227	102688227	102688227	102688227	102688227	102688227
ac.n4096.1163.shuf.csv PERFECT	203889697	203889697	203889697	203889697	203889697	203889697
adhead.n6c100.max.csv PERFECT	589368	589368	589368	589368	589368	589368
alue7065-33338-0006-082.csv PERFECT	82	82	82	82	82	82
alue7065-33338-0013-096.csv PERFECT	96	96	96	96	96	96
babyface.n6c10.max.csv PERFECT	19448	19448	19448	19448	19448	19448
BL06-camel-med.max.csv	348774508	348774508	348774208 vs 348773464 vs 348774246 vs 348773905	348774508	348774508	348774508
BL06-camel-sml.max.csv	76324399	76324399	76324399	76324399	76324399	76324160 vs 76324399
BL06-gargoyle-med.max.csv	97979938	97979938	97979304 vs 97979916 vs 97979915 vs 97979816 vs 97979892 vs 97979891 vs 97979666 vs 97979870	97979938	97979938	97979938
BL06-gargoyle-sml.max.csv	29696707	29696707	29696534	29696707	29696707	29696707 vs 29696705
bone.n6c100.max.csv PERFECT	71344	71344	71344	71344	71344	71344

Figure 5.1 Preview of the results summary page.

5.3.1 Correctness Evaluation

For each graph instance, we evaluated solver correctness over all ten runs on that instance. For each graph instance, the solver would be considered:

- **All correct:** If and only if all ten runs returned the **correct** result.

- **Deterministic wrong:** If and only if all ten runs returned the **wrong** result.
- **Non-deterministic:** If any two runs returned different results.
- **Did not finish (DNF):** If each run timed out after a 300 second threshold.

The correctness of all solvers is visually shown in figure 5.2. As alluded to in the reference solver notes, two showed inconsistent behavior. Specifically, the ECL-Maxflow GPU solver and the reference implementation based on the whitepaper we used for our solvers, PBBS-Syncpar. While this was an alarming discovery, our solver was consistent. We **do not** claim that these solvers are definitively wrong — this is simply the behavior we observed with our setup, environment, benchmarking process, and set of inputs, all of which are described in this thesis.

We can see that the Syncpar solver consistently returned correct results within the time limit roughly **81.2**% of the time. ECL did so too in **86.1**%. Other solvers were consistently correct all the time. In table 5.2, we list all the problematic graph instances and the behaviors observed for each of the two solvers.

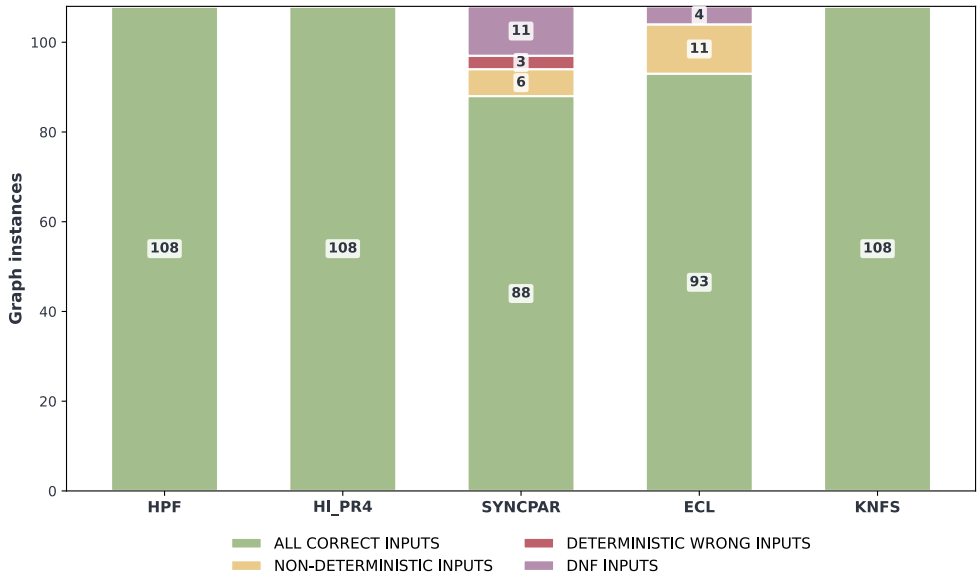


Figure 5.2 Comparison between the solvers on our set of inputs. For a graph instance to be considered correct, DNF, or deterministically wrong, said pattern must have happened during all ten runs of that instance. To be considered non-deterministic, at least two different results must be returned within the ten runs.

Table 5.2 List of graph instances on which some solvers were inconsistent. The ✓ symbol denotes that the solver was consistently correct. Solvers absent from the list were always consistently correct.

Graph Name	syncpar (3.2.4.1)	ECL (3.2.3.1)
BL06-camel-med	Non-deterministic	✓
BL06-camel-sml	✓	Non-deterministic
BL06-gargoyle-med	Non-deterministic	✓
BL06-gargoyle-sml	Consistently wrong	Non-deterministic
BVZ-sawtooth10	Consistently wrong	✓
BVZ-tsukuba14	Non-deterministic	✓
BVZ-tsukuba5	Non-deterministic	✓
BVZ-venus0	Non-deterministic	✓
car_16bins.inc	DNF	Non-deterministic, DNF
car_32bins.inc	DNF	Non-deterministic, DNF
gardenInc0001L4_L3.inc	DNF	Non-deterministic
gardenInc0003L4_L1.inc	DNF	Non-deterministic
gardenInc0004L4_L0.inc	DNF	Non-deterministic
gardenInc0004L4_L2.inc	DNF	Non-deterministic
gardenInc0005L4_L0.inc	DNF	Non-deterministic
gardenInc0005L4_L2.inc	DNF	Non-deterministic
KZ2-sawtooth1	Consistently wrong	✓
KZ2-venus6	Non-deterministic	Non-deterministic
pedestrians.inc	DNF	DNF
videoSegA2.inc	DNF	DNF
videoSegB2.inc	DNF	DNF
videoSegB.inc	DNF	DNF

5.3.2 Runtime comparison

For each graph instance, we computed the average runtime of each solver from all its runs. Since each solver, including ours, came with a timer baked in and reported its runtime, we used those time reports. Technically, this does not capture some overhead associated with launching a GPU solver executable, but we argue that this occurs once upon the executable's launch and is easily mitigated the moment you process more inputs or do multiple reruns per one executable launch, which is expected for use in practice.

In table A.1, we list the average runtime of each solver on each graph instance. Bolded entries are the best performing, consistently correct average runtime on that graph instance. This is why for some graphs, specifically some from table 5.2, the bolded entry is actually not the lowest solver runtime on that input.

The same results, in relative terms, are shown in table A.2, where we present the performance of solvers on each input relative to the best performing correct one for said input. This expresses a **slowdown** of other solvers compared to the best performing one, so 10 here means the solver is 10 times

slower than the best one on that graph instance. From this relative comparison, we can evaluate the overall performance of all solvers on our set of inputs, as shown in figure 5.3.

In our benchmark, the best performing solver was the highest-label push-relabel solver, which dominated on **50.9**% of graph instances. Then, ECL-Maxflow and our GPU backend compiled solver followed with **20.4**% and **17.6**%, respectively. PBBS-Syncpar was the best performing on **8.3**% of inputs and hpf on **2.8**%. Our CPU backend compiled solver was never the best performing one on any input graph.

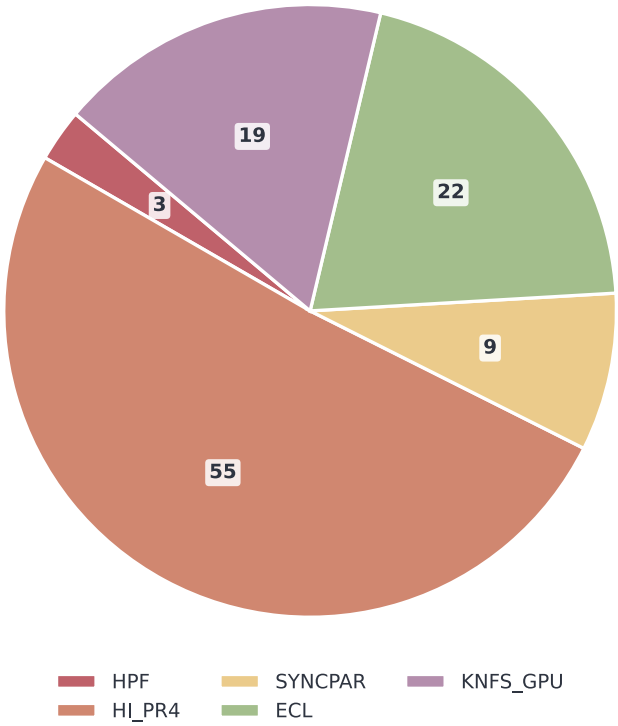


Figure 5.3 Distribution of best solver performance on our set of inputs. For a solver to be best performant on any graph instance, it must not only have the lowest average runtime, but also be consistently correct on that graph instance (see figure 5.2).

5.3.3 Performance on different backends

Because Kokkos allows our solver to be compiled for different backends and architectures, we wanted to compare the CPU compiled variant with the GPU one. We would like to note that when designing the solver, GPU was the

primary target considered.

When implementing the solver, we purposefully added timers to track three major phases of our solver pipeline: **I/O**, **Build**, and **Algorithm** phases. These match the **Loading**, **Building**, and **Solving** phases in figure 4.1.

In figure 5.4, we can see the average percentage of total runtime spent in each of these phases on each backend separately. Looking at the GPU variant, we can see a practically ideal distribution, as almost all of the time (**93**.**94**%) is spent in the algorithm phase of the solver. This means that the other two phases, which are necessary but do not contribute to solving the problem, are getting out of the way as much as possible in layman's terms. Alas, the CPU variant does not have such an ideal distribution, spending a considerable amount of time in the building phase (**27**.**65**%).

This can be a hint for future work, pointing to the build phase as a place where specialization or branching based execution space (Host or Device) might be worth considering. On the other hand, we feel that this could potentially go against the main reason we used Kokkos in the first place. Creating specialized compile time branches for specific backends reintroduces the problem of having to maintain and write multiple codebases to some, albeit lesser, extent.

Another metric to consider is the actual utilization of Kokkos. In other words, how much time is actually being spent inside these kernels, and not outside or on the overhead associated with them. This is where the kernel timer comes in, as it reports both the percentage of total runtime spent in Kokkos kernels and lists all called kernels with the amount of in-kernel time spent in them. Table A.3 shows this information for both CPU and GPU backends. We can see that Kokkos utilization varies greatly, even among graphs that should have similar topology, such as `luxembourg.osm` and `europe.osm`. Averaging each solver Kokkos utilization over our benchmark set shows that overall, their utilization is actually very similar. The GPU backend solver spends, on average, **74**.**16**% of its total runtime in Kokkos kernels, while the CPU does so in **74**.**23**%. This bounds the framework overhead of using Kokkos at roughly **26**% in the worst case, where all time outside of kernels is Kokkos related overhead. Realistically, it is smaller than this.

For the majority of graph instances, the process kernel was the main hotspot where most of the in-kernel time was spent, followed by global relabel, which seems to be more relevant on the GPU than on the CPU.

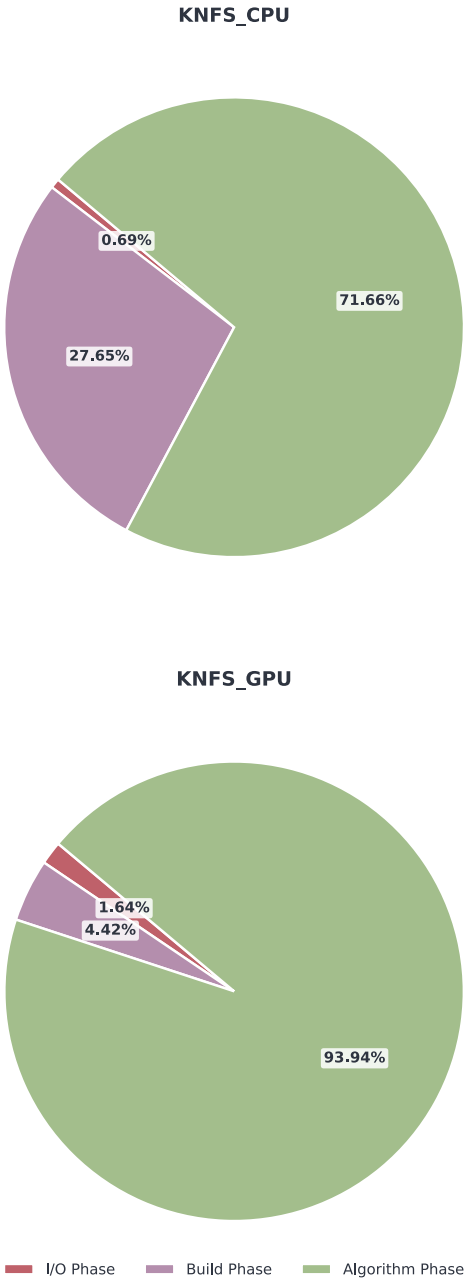


Figure 5.4 Average time distribution across execution phases for CPU and GPU backends, averaged out from all runs on our set of inputs.

Chapter 6

Issues & Potential improvements

In this chapter, we would like to comment on some current issues regarding our implementation, as well as outline and note directions and actions that could be taken to further improve, fix, or expand it. These are things that were considered or overlooked but could not be fully explored or fleshed out due to being outside the scope of our work.

6.1 Issues

6.1.1 Issue: Flow Sloshing

While our implementation is empirically correct, we observed instances of heavy redundant work known as *flow sloshing*. When relying purely on local relabels, adjacent active nodes can enter a state where they continuously push flow back and forth, slowly raising their height labels without making global progress. To our knowledge, this is a known algorithmic inefficiency of the Push-Relabel methodology, stemming from the locality of the operations. In practice, this pathology is mitigated by our global relabel step, but when disabled or when the threshold is too high, the solver's iteration count spikes dramatically.

For a follow-up work, this would be an important aspect to fully explore and clarify. It is important to try to mitigate this as much as possible — for instance, tying this to our suggestion of a dynamic global relabel trigger (see subsection 6.2.3) and tuning it based on the findings.

6.1.2 Issue: Hybrid approach GPU instability

Similar to the ECL-Maxflow solver, when we ran our degree-based hybrid implementation on our benchmark set of inputs, it seemed to be unstable, occasionally crashing GPU in the same way (see subsection 3.2.3.1). Note that it was consistently correct when it did not crash. Luckily, our primary implementation does not suffer from these issues.

6.2 Potential improvements

6.2.1 Further analyze building process for CPU

As we noted in subsection 5.3.3, running our solver on the CPU backend seems to spend a considerable amount of time in the graph building phase, especially compared to the GPU (see figure 5.4). At first, we thought this might have something to do with memory usage, which was supported by the high-water mark (peak memory usage during runtime) being **three times higher** on average on the CPU solver. This turned out to be a red herring, as the profiling tool seems to only measure system memory, so GPU allocations were never recorded by it. To confirm, we ran the Kokkos memory usage tool that recorded all memory spaces. It showed that the memory usage was not significantly higher, and both solvers behaved relatively similarly in terms of their memory footprint. The CPU solver still had a slightly higher peak usage, but that makes sense, as the GPU solver had its memory usage split between system memory and VRAM. Adding up the memory allocated in different memory spaces was roughly equal to the CPU solver's peak, confirming they behaved the same.

Nevertheless, this warrants further analysis to determine whether and how to improve this part of the pipeline to better fit the premise of the Kokkos framework. An oversight on our part and a clear next step would be to include **regions** in our solver. Kokkos region is a scope based mechanism, allowing for scoping out and splitting the profiling of code into regions. This would allow the kernel timer profiling tool to be far more granular, in turn letting us analyze the building phase in greater detail.

6.2.2 Memory traits integration

Another avenue for optimization would be to try to integrate Kokkos memory traits, which we discussed in subsection 2.2.2.1. For instance, specifying CSR graph representation views as random access. We believe this would not cause any major issues if the specified trait aligns with the expected use of the view.

6.2.3 Dynamic global relabel trigger

Currently, our implementation has a fixed trigger for global relabeling. This is a poor man's solution that, by its design, cannot universally scale and adapt to different input topologies. We initially had a work based threshold akin to that of the reference PRSN algorithm, but it proved to be too high, especially given the potential sloshing issue (see subsection 6.1.1).

Furthermore, as mentioned in [27], it seems that generic GPU oriented push relabel solvers tie their threshold to the number of pushes or iterations rather than the flow moved, as in PRSN.

To allow for a more adaptive global relabel trigger, a heuristic could be introduced to set it based on input specific details, probably the number of vertices or edge density. For instance, ECL-Maxflow uses $\frac{|V|^2}{1000 \times |E|}$.

6.2.4 Hybrid global relabel

Currently, our global relabel uses frontier based BFS with queues. This approach forces the use of atomics when enqueueing vertices. At some point during the BFS, many vertices can be trying to enqueue at once, possibly leading to atomic contention. An alternative approach would be to use iteration or boolean masks, where we could set a mask element corresponding to our vertex without atomics. This has the downside of forcing a kernel launch over all vertices, even if only a minuscule fraction will be active.

To get the best of both worlds, a direction optimized BFS, as described in [33], could be adapted and would almost certainly lead to a speedup, especially given the importance of global relabeling within our solver.

6.2.5 Bandwidth reduction & Vertex reordering

Currently, our implementation loads the graph as presented in the input file without any specific reordering step. While our graph representation design allows for accessing edges in contiguous, coalesced memory reads, this is not the case for neighboring vertex properties, such as the label or excess.

Since the algorithm is memory bandwidth-bound, as noted by Baumstark et al. in [11], it stands to reason that improving spatial locality via some pre-processing pass (e.g., Reverse Cuthill-McKee [34]) could improve overall performance. This would likely be felt most in the global relabel step, as it would improve the cache hit rate at the frontiers. But given that our target use case for the solver is a one-time calculation on the input graph, the overhead of this processing pass would most likely eat up any gains.

Where this becomes a worthwhile endeavor, at the very least, is in dynamic max flow scenarios. Suppose the graph topology is fixed for a whole batch of inputs, but some properties change over time. This is a rather common scenario in computer vision, for instance in image segmentation and restoration. In such use cases, the overhead of the processing pass is effectively amortized. On the other hand, when these domain specifics are known beforehand, a better practical option would often be to choose a highly specialized approach, such as the Boykov-Kolmogorov algorithm [35].

6.2.6 Hierarchical Parallelism

While we did explore the use of more complex Kokkos features to model an alternative approach utilizing hierarchical parallelism over both vertices and

edges, this was more of a detour and a proof-of-concept for the sake of correctness and feasibility rather than a complete devotion to this idea. We believe it shows that such a design could certainly be realized using Kokkos. It might be worth building and fully experimenting with a solver based on this approach.

Conclusion

To reiterate, the goal of this thesis was to investigate the feasibility and performance characteristics of implementing graph algorithms, specifically the maximum flow problem, using performance portability frameworks, particularly Kokkos. The ultimate aim was to see if graph problems, which are often irregular and hard to map onto modern manycore hardware, could be implemented in a device-agnostic manner, resulting in a single codebase that could be compiled for different architectures. This has a plethora of benefits, even more so with the undeniable ongoing shift to heterogeneous architectures in HPC.

As our main objective, we have set out to create a maximum flow problem solver based on a parallel synchronous push-relabel approach, implemented in Kokkos and capable of running on both CPU and GPU architectures. We firmly believe that we have achieved this by creating a fully capable, consistent, and respectable KNFS solver implementation. In addition to our primary solver, we also proposed and implemented a naive but functional proof-of-concept alternative approach. There, we experimented with more complex Kokkos features to demonstrate its ability to leverage and potentially enable the hierarchical parallelism of modern architectures further.

Another objective was to perform an evaluation of a representative set of 108 graph instances, along with alternative reference solvers representing various architectures and approaches. Through this testing, our solver demonstrated empirical correctness, consistency, and competitive performance. Notably, while some reference implementations, such as ECL-Maxflow and PBBS-Syncpar, exhibited non-deterministic behavior or crashed on specific edge-case inputs, our solver remained correct.

When examining performance characteristics, we also focused on aspects that are unique to our solver: Kokkos framework utilization and the framework overhead incurred on our device-agnostic code. While having a singular codebase for many backends is extremely beneficial, we have noticed that not all backends are absolutely equal. On the GPU, our solver spent 93.94% of its total runtime in the *productive* computational part of the code, meaning that

the rest of the pipeline (loading and parsing of the graph and building it) consumed a fraction of the runtime. The same cannot be said for the CPU, where this percentage dropped to roughly 71.66% due to the graph building phase. While this does make sense, as we designed it with a massive GPU thread count in mind and it can certainly out-scale the CPU, it must be noted. Using a less *GPU-biased* implementation would probably lead to more consistent performance across all backends.

A kernel timer analysis revealed that our solver, no matter the backend, spends roughly 74% of its total runtime inside Kokkos kernels. From this, we can determine 26% as the upper bound estimate of the framework overhead for using Kokkos in our code. Let us now assume that our code is flawless (it most definitely is **not**), that this estimate cannot be lowered, and that all time spent outside kernels is directly tied to Kokkos overhead (it most definitely is **not**). Even in this extreme scenario with a direct 26% performance cut, we would still consider it a worthy trade-off for a singular codebase deployable on the majority of HPC hardware configurations, only needing a recompile.

Do note that irregular graph algorithms represent a rather difficult class of problems to map efficiently onto any single architecture, let alone to abstract across multiple backends simultaneously. For more specialized problems, such as those with fixed topology (e.g., image segmentation), optimizations based on this added context could be implemented with Kokkos, translating to the backends. This is to say that one could realistically achieve a much smaller performance cut on the majority of more specific problems, or so we firmly believe.

As with any scientific endeavor, this work has clear avenues for future optimization. During our evaluation, we identified an issue of *flow sloshing* — a phenomenon of redundant local relabeling between adjacent vertices without making global progress. Fully diagnosing and mitigating this, alongside replacing our fixed global relabel trigger with an adaptive, input-specific heuristic based on metrics like edge density, represents the immediate next steps for further improving performance.

Ultimately, this thesis confirms and demonstrates the viability of using performance portability frameworks, even for irregular graph algorithms. The engineering hours and redundant work saved by having a singular codebase able to run on practically any standard HPC configuration are substantial. This approach allows developers to reallocate their regained time toward algorithmic improvements rather than platform-specific tuning and maintenance. What's more, relying on a framework-level abstraction keeps the codebase less prone to architecture-specific shifts and dependencies.

Appendix A

Tables

A.1 Average runtimes

Table A.1 Average runtimes on our set of inputs in seconds. Times are averaged from ten runs. Entries in bold are **the fastest average runtime with constistent correct result** for a given graph instance.

Graph Name	hpf [s]	hipr4 [s]	syncpar [s]	knfs cpu [s]	knfs gpu [s]	ECL [s]
europe.osm	50.4046	**1.7452**	7.3837	42.9286	2.5312	15.0626
ac.n1024.1163.shuf	0.1567	**0.0302**	0.0592	0.3503	1.2208	0.0528
ac.n2048.1163.shuf	0.6229	0.1356	**0.0615**	0.6885	1.3478	0.1262
ac.n4096.1163.shuf	3.0605	0.8818	**0.1649**	2.7613	6.9851	0.4725
adhead.n6c100	54.0280	22.1593	13.8385	30.3378	**2.4975**	5.8873
alue7065-33338-0006-082	0.0433	**0.0236**	0.3216	1.3490	0.8313	0.3048
alue7065-33338-0013-096	0.0444	**0.0250**	0.3283	1.3256	0.8566	0.3091
babyface.n6c10	24.6454	14.1532	9.5664	18.5377	**1.1867**	2.7066
BL06-camel-med	36.9050	36.8286	16.1392	39.8652	13.7128	**7.5290**
BL06-camel-sml	3.1667	1.5898	**1.1529**	6.2040	2.2859	0.8441
BL06-gargoyle-med	28.9836	24.6422	27.3429	38.0880	**5.6815**	6.3238
BL06-gargoyle-sml	2.4723	**1.0241**	1.3385	7.8773	1.5195	0.7268
bone.n6c100	16.3406	4.8910	4.8961	17.1945	**1.7201**	3.6380
bone.n6c10	14.6326	4.9168	6.3352	16.0622	**1.7935**	3.5330
bone_subx.n26c10	0.4925	0.1599	**0.0888**	0.6823	0.2601	0.1982
bone_subxy.n6c10	3.8262	0.7293	1.3031	3.8087	**0.7082**	0.8543
bone_subxyz.n26c100	5.1872	0.8979	1.3238	5.2939	**0.5135**	0.9660
bone_subxyz.n6c100	1.9304	0.3233	0.7192	2.0703	**0.3100**	0.4766
bone_subxyz.n6c10	1.8111	**0.3018**	0.7373	2.0652	0.4197	0.4872
bone_subxyz_subx.n6c100	0.9154	**0.1376**	0.4819	1.1095	0.2718	0.2706
bug	1.7507	**0.1310**	0.2830	8.2436	0.4259	0.2615
BVZ-sawtooth0	0.4702	0.1550	0.2577	8.3277	0.4522	**0.1452**
BVZ-sawtooth10	0.5598	0.2185	0.3589	7.1013	0.4760	**0.1506**
BVZ-sawtooth4	0.4446	**0.1149**	0.2287	7.1203	0.3466	0.1712
BVZ-tsukuba14	0.2717	**0.0877**	0.2375	5.1123	0.2848	0.1218
BVZ-tsukuba5	0.2089	**0.0644**	0.2169	2.4846	0.2504	0.1772
BVZ-venus0	0.5241	0.2506	0.5135	6.3616	0.3486	**0.1470**
BVZ-venus7	0.5328	**0.2552**	1.0825	5.5102	0.5776	0.3471
BVZ-venus9	0.5652	**0.2769**	0.6136	5.5353	0.4089	0.3193
cal-1800721-0008-018	4.9626	4.8120	7.9865	3.3600	2.0474	**1.5182**
cal-1800721-0010-037	5.4808	4.2832	7.4322	5.4006	**3.0302**	3.3617
car_16bins	0.2623	**0.0679**	0.2279	1.0727	0.9039	0.3186
car_16bins.inc	0.6090	**0.1519**	*DNF*	1.1758	1.7902	1.2968
car_32bins.inc	0.6146	**0.1348**	*DNF*	1.2478	1.3665	0.9952
car_64bins	0.2681	**0.0692**	0.2108	1.1692	0.4129	0.1853
comp2	2.3512	**0.3126**	1.4053	8.9553	0.5410	0.5041
cowInc0002L4_L1.inc	0.6296	0.1887	**0.0807**	0.4538	10.6400	0.1568
cowInc0003L4_L0.inc	1.0230	0.2213	**0.0754**	0.8031	14.7833	0.2257
cowInc0004L4_L2.inc	0.9565	0.2094	**0.0820**	0.6564	17.3889	0.2504
cowInc0005L4_L2.inc	0.9623	0.2103	**0.0737**	0.6810	17.4322	0.2499
cowInc0005L4_L3.inc	1.0419	0.2197	**0.0899**	0.8033	16.8647	0.2337

Continued on next page

Table A.1 (A.1 Continued)

Graph Name	hpf [s]	hipr4 [s]	syncpar [s]	knfs cpu [s]	knfs gpu [s]	ECL [s]
crab	2.3935	0.4775	0.9740	11.4208	0.6817	**0.4729**
flamingo2	8.8741	0.6528	1.7288	15.9798	1.0944	**0.6411**
gardenInc0001L4_L3.inc	0.0324	**0.0021**	*DNF*	0.0402	0.1355	0.0195
gardenInc0002L4_L2.inc	0.0461	**0.0026**	0.0565	0.0533	0.1883	0.0162
gardenInc0003L4_L1.inc	0.0412	**0.0025**	*DNF*	0.0472	0.1595	0.0146
gardenInc0003L4_L3.inc	0.0339	**0.0023**	0.0586	0.0425	0.1483	0.0199
gardenInc0004L4_L0.inc	0.0353	**0.0022**	*DNF*	0.0434	0.1556	0.0144
gardenInc0004L4_L2.inc	0.0467	**0.0024**	0.0517	0.0473	0.1608	0.0176
gardenInc0005L4_L0.inc	0.0342	**0.0022**	*DNF*	0.0364	0.1440	0.0160
gardenInc0005L4_L2.inc	0.0475	**0.0025**	*DNF*	0.0508	0.1610	0.0167
horse-48112-0100-120	0.1597	**0.0726**	0.5190	0.6947	0.4783	0.1954
KZ2-sawtooth19	1.0961	0.5087	0.3390	13.6528	1.0690	**0.2146**
KZ2-sawtooth1	1.1030	0.5331	0.2725	14.1962	1.0708	**0.2089**
KZ2-venus18	1.2593	0.6795	0.5003	10.9326	1.2565	**0.2379**
KZ2-venus2	1.3307	0.8718	0.4610	11.4662	1.3732	**0.2380**
KZ2-venus6	0.9143	**0.3821**	0.3287	6.2115	1.1658	0.2440
lazybrush-bird	19.1962	11.2342	122.3500	18.7821	**5.6535**	8.6662
lazybrush-doctor	5.4118	6.9855	23.2411	9.5999	**1.8270**	2.3961
lazybrush-elephant	21.6779	14.3869	98.9728	17.0404	**4.4867**	10.8348
lazybrush-footman	1.1700	0.8089	1.7308	2.2671	**0.4981**	0.8735
lazybrush-hmdman	1.5418	1.1902	3.6005	3.7064	**0.5924**	1.0114
lazybrush-mangadinner	2.1450	2.6982	22.6487	6.7200	**1.7528**	2.0239
lazybrush-mangagirl	**1.4781**	1.8681	16.9379	8.3175	4.0666	3.5024
LB07-bunny-med	18.6076	22.6876	6.3065	42.0737	4.3254	**3.4160**
LB07-bunny-sml	2.0911	1.0758	0.8089	7.2698	0.8489	**0.4655**
liver.n26c10	16.8550	8.0653	6.1947	14.0154	**1.2116**	2.0661
liver.n6c100	19.4463	11.0203	6.8755	14.9556	**1.1994**	2.0377
lux-108726-0033-009	**0.0922**	0.1345	1.0245	2.1292	1.1551	1.2102
lux-112498-0010-010	**0.1016**	0.1058	0.9946	1.0100	0.6510	0.7491
luxembourg.osm	0.1061	**0.0025**	0.0841	0.4412	0.2766	0.0976
nj2010-165230-0018-143	0.3551	**0.0886**	0.4322	2.4157	0.5346	0.1790
nj2010-165230-0032-166	0.3907	**0.1784**	0.4904	1.0956	0.7949	0.2488
nl-2159462-0031-021	3.3514	5.4328	6.8906	5.2319	2.6849	**2.1333**
nl-2159462-0276-041	2.7861	5.8587	10.5018	4.2232	**1.3783**	1.4374
pedestrians.inc	7.1422	**0.2968**	*DNF*	7.3920	1.0488	*DNF*
person_16bins	0.3059	**0.0553**	0.1973	0.7902	0.5376	0.7466
person_64bins	0.3112	**0.0542**	0.2020	0.7932	0.3547	0.1261
punch-eu18-05-p	0.2269	**0.1551**	0.8004	1.5276	0.5608	0.2738
punch-eu20-08-u	0.6452	0.5372	1.2020	2.6255	1.2172	**0.5019**
punch-eu20-09-p	0.8666	**0.5230**	1.6262	2.7726	0.8172	0.5557
punch-eu22-06-p	8.0896	5.2506	6.8170	7.1143	**1.5856**	1.6651
punch-us18-01-u	0.0762	**0.0078**	0.1614	0.7139	0.4162	0.1165
punch-us18-07-u	0.1812	**0.0850**	0.4722	1.5823	0.8142	0.3986
punch-us18-08-p	0.1609	**0.0544**	0.9380	1.1638	0.5756	0.3598
punch-us18-10-p	0.1113	**0.0315**	0.5213	0.8983	0.4666	0.2464
punch-us20-03-p	0.4785	**0.1384**	1.2827	1.3980	0.5607	0.4382
punch-us20-04-u	0.5811	**0.3727**	1.2889	1.8438	0.9594	0.4658
punch-us22-07-u	3.4503	2.2098	8.1065	5.4195	2.1152	**1.4008**
punch-us22-09-u	3.0004	2.7454	11.2348	5.5187	2.4392	**1.5438**
punch-us22-10-p	7.4795	2.7052	20.3596	7.1791	2.4848	**2.0190**
rgg17-129426-0025-390	0.8342	**0.3940**	1.8783	1.7629	1.3014	0.4831
rgg18-260352-0032-319	2.3263	1.6565	2.1084	1.6232	0.9175	**0.5132**
rmf-long.n2.3142.shuf	0.1327	**0.0344**	2.9570	1.9600	1.1492	0.4697
rmf-long.n3.3142.shuf	0.3227	**0.0476**	7.0162	2.9795	1.6204	0.8935
rmf-long.n4.3142.shuf	1.0423	**0.0915**	12.7350	4.6153	2.0700	1.4104
rmf-wide.n2.3141.shuf	0.0952	**0.0752**	4.7241	8.9465	4.7383	1.2057
rmf-wide.n3.3141.shuf	0.2695	**0.1588**	11.5206	14.0713	7.6936	2.0578
rmf-wide.n4.3141.shuf	0.6897	**0.3390**	26.8503	36.2742	20.4675	3.8006
videoSegA2	0.9048	0.6096	0.6416	3.1817	0.3446	**0.2463**
videoSegA2.inc	11.8458	**1.2816**	*DNF*	5.0577	2.1705	*DNF*
videoSegA	0.9331	0.5739	0.7747	2.9656	0.3621	**0.2327**
videoSegB2.inc	3.7291	**0.1766**	*DNF*	2.5903	3.4918	*DNF*
videoSegB	0.5791	**0.0620**	0.1479	1.5746	0.7761	0.2958
videoSegB.inc	3.5363	**0.1712**	*DNF*	2.5628	3.5125	*DNF*
wash-rlg-long.n1024.1163.shuf	0.1005	**0.0346**	2.6635	1.5291	0.7717	0.9540
wash-rlg-long.n2048.1163.shuf	0.2482	**0.0595**	6.1639	2.4874	1.2631	2.2161
wash-rlg-wide.n2048.1163.shuf	0.3383	0.2168	1.4435	1.8022	0.4024	**0.1883**

A.2 Relative slowdown

Table A.2 Relative slowdown of solvers compared to the best performing consistently correct one (bolded) on a given input. '**X**' denotes solver being faster but inconsistent or wrong.

Graph Name	hpf	hipr4	syncpar	knfs cpu	knfs gpu	ECL
europe.osm	28.88	**1.00**	4.23	24.60	1.45	8.63
ac.n1024.1163.shuf	5.19	**1.00**	1.96	11.60	40.42	1.75
ac.n2048.1163.shuf	10.13	2.20	**1.00**	11.20	21.92	2.05
ac.n4096.1163.shuf	18.56	5.35	**1.00**	16.75	42.36	2.87
adhead.n6c100	21.63	8.87	5.54	12.15	**1.00**	2.36
alue7065-33338-0006-082	1.83	**1.00**	13.63	57.16	35.22	12.92
alue7065-33338-0013-096	1.78	**1.00**	13.13	53.02	34.26	12.36
babyface.n6c10	20.77	11.93	8.06	15.62	**1.00**	2.28
BL06-camel-med	4.90	4.89	2.14	5.29	1.82	**1.00**
BL06-camel-sml	2.75	1.38	**1.00**	5.38	1.98	**X**
BL06-gargoyle-med	5.10	4.34	4.81	6.70	**1.00**	1.11
BL06-gargoyle-sml	2.41	**1.00**	1.31	7.69	1.48	**X**
bone.n6c100	9.50	2.84	2.85	10.00	**1.00**	2.11
bone.n6c10	8.16	2.74	3.53	8.96	**1.00**	1.97
bone_subx.n26c10	5.55	1.80	**1.00**	7.68	2.93	2.23
bone_subxy.n6c10	5.40	1.03	1.84	5.38	**1.00**	1.21
bone_subxyz.n26c100	10.10	1.75	2.58	10.31	**1.00**	1.88
bone_subxyz.n6c100	6.23	1.04	2.32	6.68	**1.00**	1.54
bone_subxyz.n6c10	6.00	**1.00**	2.44	6.84	1.39	1.61
bone_subxyz_subx.n6c100	6.65	**1.00**	3.50	8.06	1.98	1.97
bug	13.36	**1.00**	2.16	62.93	3.25	2.00
BVZ-sawtooth0	3.24	1.07	1.77	57.35	3.11	**1.00**
BVZ-sawtooth10	3.72	1.45	2.38	47.15	3.16	**1.00**
BVZ-sawtooth4	3.87	**1.00**	1.99	61.97	3.02	1.49
BVZ-tsukuba14	3.10	**1.00**	2.71	58.29	3.25	1.39
BVZ-tsukuba5	3.24	**1.00**	3.37	38.58	3.89	2.75
BVZ-venus0	3.57	1.70	3.49	43.28	2.37	**1.00**
BVZ-venus7	2.09	**1.00**	4.24	21.59	2.26	1.36
BVZ-venus9	2.04	**1.00**	2.22	19.99	1.48	1.15
cal-1800721-0008-018	3.27	3.17	5.26	2.21	1.35	**1.00**
cal-1800721-0010-037	1.81	1.41	2.45	1.78	**1.00**	1.11
car_16bins	3.86	**1.00**	3.36	15.80	13.31	4.69
car_16bins.inc	4.01	**1.00**	*DNF*	7.74	11.79	8.54
car_32bins.inc	4.56	**1.00**	*DNF*	9.26	10.14	7.38
car_64bins	3.87	**1.00**	3.05	16.90	5.97	2.68
comp2	7.52	**1.00**	4.50	28.65	1.73	1.61
cowInc0002L4_L1.inc	7.80	2.34	**1.00**	5.62	131.85	1.94
cowInc0003L4_L0.inc	13.57	2.94	**1.00**	10.65	196.06	2.99
cowInc0004L4_L2.inc	11.66	2.55	**1.00**	8.00	212.06	3.05
cowInc0005L4_L2.inc	13.06	2.85	**1.00**	9.24	236.53	3.39
cowInc0005L4_L3.inc	11.59	2.44	**1.00**	8.94	187.59	2.60
crab	5.06	1.01	2.06	24.15	1.44	**1.00**
flamingo2	13.84	1.02	2.70	24.93	1.71	**1.00**
gardenInc0001L4_L3.inc	15.43	**1.00**	*DNF*	19.14	64.52	9.29
gardenInc0002L4_L2.inc	17.73	**1.00**	21.73	20.50	72.42	6.23
gardenInc0003L4_L1.inc	16.48	**1.00**	*DNF*	18.88	63.80	5.84
gardenInc0003L4_L3.inc	14.74	**1.00**	25.48	18.48	64.48	8.65
gardenInc0004L4_L0.inc	16.05	**1.00**	*DNF*	19.73	70.73	6.55
gardenInc0004L4_L2.inc	19.46	**1.00**	21.54	19.71	67.00	7.33
gardenInc0005L4_L0.inc	15.55	**1.00**	*DNF*	16.55	65.45	7.27
gardenInc0005L4_L2.inc	19.00	**1.00**	*DNF*	20.32	64.40	6.68
horse-48112-0100-120	2.20	**1.00**	7.15	9.57	6.59	2.69
KZ2-sawtooth19	5.11	2.37	1.58	63.62	4.98	**1.00**
KZ2-sawtooth1	5.28	2.55	1.30	67.96	5.13	**1.00**
KZ2-venus18	5.29	2.86	2.10	45.95	5.28	**1.00**
KZ2-venus2	5.59	3.66	1.94	48.18	5.77	**1.00**
KZ2-venus6	2.39	**1.00**	**X**	16.26	3.05	**X**
lazybrush-bird	3.40	1.99	21.64	3.32	**1.00**	1.53
lazybrush-doctor	2.96	3.82	12.72	5.25	**1.00**	1.31
lazybrush-elephant	4.83	3.21	22.06	3.80	**1.00**	2.41
lazybrush-footman	2.35	1.62	3.47	4.55	**1.00**	1.75
lazybrush-hmdman	2.60	2.01	6.08	6.26	**1.00**	1.71
lazybrush-mangadinner	1.22	1.54	12.92	3.83	**1.00**	1.15
lazybrush-mangagirl	**1.00**	1.26	11.46	5.63	2.75	2.37
LB07-bunny-med	5.45	6.64	1.85	12.32	1.27	**1.00**
LB07-bunny-sml	4.49	2.31	1.74	15.62	1.82	**1.00**
liver.n26c10	13.91	6.66	5.11	11.57	**1.00**	1.71
liver.n6c100	16.21	9.19	5.73	12.47	**1.00**	1.70
lux-108726-0033-009	**1.00**	1.46	11.11	23.09	12.53	13.13

Continued on next page

Table A.2 (A.2 Continued)

Graph Name	hpf	hipr4	syncpar	knfs cpu	knfs gpu	ECL
lux-112498-0010-010	**1.00**	1.04	9.79	9.94	6.41	7.37
luxembourg.osm	42.44	**1.00**	33.64	176.48	110.64	39.04
nj2010-165230-0018-143	4.01	**1.00**	4.88	27.27	6.03	2.02
nj2010-165230-0032-166	2.19	**1.00**	2.75	6.14	4.46	1.39
nl-2159462-0031-021	1.57	2.55	3.23	2.45	1.26	**1.00**
nl-2159462-0276-041	2.02	4.25	7.62	3.06	**1.00**	1.04
pedestrians.inc	24.06	**1.00**	*DNF*	24.91	3.53	*DNF*
person_16bins	5.53	**1.00**	3.57	14.29	9.72	13.50
person_64bins	5.74	**1.00**	3.73	14.63	6.54	2.33
punch-eu18-05-p	1.46	**1.00**	5.16	9.85	3.62	1.77
punch-eu20-08-u	1.29	1.07	2.39	5.23	2.43	**1.00**
punch-eu20-09-p	1.66	**1.00**	3.11	5.30	1.56	1.06
punch-eu22-06-p	5.10	3.31	4.30	4.49	**1.00**	1.05
punch-us18-01-u	9.77	**1.00**	20.69	91.53	53.36	14.94
punch-us18-07-u	2.13	**1.00**	5.56	18.62	9.58	4.69
punch-us18-08-p	2.96	**1.00**	17.24	21.39	10.58	6.61
punch-us18-10-p	3.53	**1.00**	16.55	28.52	14.81	7.82
punch-us20-03-p	3.46	**1.00**	9.27	10.10	4.05	3.17
punch-us20-04-u	1.56	**1.00**	3.46	4.95	2.57	1.25
punch-us22-07-u	2.46	1.58	5.79	3.87	1.51	**1.00**
punch-us22-09-u	1.94	1.78	7.28	3.57	1.58	**1.00**
punch-us22-10-p	3.70	1.34	10.08	3.56	1.23	**1.00**
rgg17-129426-0025-390	2.12	**1.00**	4.77	4.47	3.30	1.23
rgg18-260352-0032-319	4.53	3.23	4.11	3.16	1.79	**1.00**
rmf-long.n2.3142.shuf	3.86	**1.00**	85.96	56.98	33.41	13.65
rmf-long.n3.3142.shuf	6.78	**1.00**	147.40	62.59	34.04	18.77
rmf-long.n4.3142.shuf	11.39	**1.00**	139.18	50.44	22.62	15.41
rmf-wide.n2.3141.shuf	1.27	**1.00**	62.82	118.97	63.01	16.03
rmf-wide.n3.3141.shuf	1.70	**1.00**	72.55	88.61	48.45	12.96
rmf-wide.n4.3141.shuf	2.03	**1.00**	79.20	107.00	60.38	11.21
videoSegA2	3.67	2.48	2.60	12.92	1.40	**1.00**
videoSegA2.inc	9.24	**1.00**	*DNF*	3.95	1.69	*DNF*
videoSegA	4.01	2.47	3.33	12.74	1.56	**1.00**
videoSegB2.inc	21.12	**1.00**	*DNF*	14.67	19.77	*DNF*
videoSegB	9.34	**1.00**	2.39	25.40	12.52	4.77
videoSegB.inc	20.66	**1.00**	*DNF*	14.97	20.52	*DNF*
wash-rlg-long.n1024.1163.shuf	2.90	**1.00**	76.98	44.19	22.30	27.57
wash-rlg-long.n2048.1163.shuf	4.17	**1.00**	103.59	41.81	21.23	37.25
wash-rlg-wide.n2048.1163.shuf	1.80	1.15	7.67	9.57	2.14	**1.00**

A.3 Kokkos utilization

Table A.3 Kernel timer results of our solver on CPU and GPU. *Kokkos %* shows percentage of time spent in Kokkos kernels out of the entire runtime. *Hotspot* points to the kernel where solver spent the most of in-kernel time. Last column is the percentage of in-kernel time that was spent in the hotspot. **GR** stands for global relabel, **PK** for process kernel, and **AK** for apply kernel.

	KNFS CPU			KNFS GPU		
Graph	Kokkos %	Hotspot	%	Kokkos %	Hotspot	%
europe.osm	23.96%	GR	63.08%	49.07%	GR	75.98%
ac.n1024.1163.shuf	67.00%	PK	47.03%	94.11%	PK	97.08%
ac.n2048.1163.shuf	37.08%	PK	42.60%	93.76%	PK	96.01%
ac.n4096.1163.shuf	27.32%	PK	42.31%	97.37%	PK	96.05%
adhead.n6c100	47.44%	PK	51.78%	74.88%	GR	54.18%
alue7065-33338-0006-082	94.32%	PK	46.76%	56.93%	PK	67.08%
alue7065-33338-0013-096	94.02%	PK	44.82%	55.81%	PK	65.25%
babyface.n6c10	62.85%	PK	58.16%	64.78%	PK	66.21%
BL06-camel-med	79.74%	PK	62.09%	96.64%	GR	65.51%
BL06-camel-sml	84.98%	PK	69.88%	92.52%	GR	58.17%
BL06-gargoyle-med	80.85%	PK	62.48%	92.42%	PK	58.72%
BL06-gargoyle-sml	89.06%	PK	70.43%	88.98%	PK	61.48%
bone.n6c10	41.94%	PK	38.87%	67.09%	GR	54.88%
bone.n6c100	42.06%	PK	45.06%	66.87%	GR	50.17%
bone_subx.n26c10	64.73%	PK	40.98%	66.80%	GR	63.77%
bone_subxy.n6c10	43.14%	PK	44.78%	64.19%	GR	40.08%
bone_subxyz.n26c100	34.13%	PK	48.04%	67.21%	PK	68.75%
bone_subxyz.n6c10	49.81%	PK	51.51%	61.12%	PK	53.69%
bone_subxyz.n6c100	50.40%	PK	54.09%	63.16%	PK	56.23%
bone_subxyz_subx.n6c100	54.35%	PK	53.07%	66.95%	PK	60.28%
bug	96.95%	PK	74.07%	81.15%	PK	70.94%
BVZ-sawtooth0	98.27%	PK	77.09%	71.69%	PK	68.57%
BVZ-sawtooth10	98.01%	PK	74.58%	72.02%	PK	70.25%
BVZ-sawtooth4	98.22%	PK	75.16%	78.77%	PK	70.03%
BVZ-tsukuba14	98.21%	PK	76.07%	74.05%	PK	68.21%
BVZ-tsukuba5	96.58%	PK	70.74%	70.63%	PK	61.67%
BVZ-venus0	98.02%	PK	76.05%	78.68%	PK	72.02%
BVZ-venus7	97.15%	PK	73.39%	65.49%	PK	62.13%
BVZ-venus9	97.38%	PK	75.29%	70.57%	PK	67.88%
cal-1800721-0008-018	75.48%	GR	42.69%	69.81%	PK	59.31%
cal-1800721-0010-037	83.63%	GR	49.96%	63.67%	PK	44.86%
car_16bins	85.59%	PK	62.54%	91.97%	PK	90.36%
car_16bins.inc	69.94%	PK	63.09%	95.18%	PK	94.78%
car_32bins.inc	73.34%	PK	63.21%	94.08%	PK	93.36%
car_64bins	87.21%	PK	64.23%	82.28%	PK	76.61%
comp2	95.76%	PK	75.89%	83.98%	PK	72.84%
cowInc0002L4_L1.inc	29.43%	PK	27.02%	99.72%	PK	99.80%
cowInc0003L4_L0.inc	18.75%	PK	25.58%	99.75%	PK	99.84%
cowInc0004L4_L2.inc	21.50%	PK	24.91%	99.78%	PK	99.87%
cowInc0005L4_L2.inc	19.97%	PK	23.30%	99.80%	PK	99.87%
cowInc0005L4_L3.inc	17.76%	PK	24.07%	99.79%	PK	99.88%
crab	95.83%	PK	78.83%	87.05%	PK	78.33%
flamingo2	95.00%	PK	78.88%	89.86%	PK	79.15%
gardenInc0001L4_L3.inc	49.14%	PK	27.45%	93.28%	PK	87.14%
gardenInc0002L4_L2.inc	39.29%	PK	23.13%	95.32%	PK	91.53%
gardenInc0003L4_L1.inc	41.57%	PK	32.05%	93.35%	PK	89.83%
gardenInc0003L4_L3.inc	53.22%	PK	30.49%	93.34%	PK	88.94%
gardenInc0004L4_L0.inc	39.84%	PK	28.24%	94.82%	PK	89.52%
gardenInc0004L4_L2.inc	42.75%	PK	27.02%	93.78%	PK	90.27%
gardenInc0005L4_L0.inc	41.75%	PK	25.41%	95.86%	PK	89.85%
gardenInc0005L4_L2.inc	37.81%	PK	28.00%	93.85%	PK	90.86%
horse-48112-0100-120	90.88%	PK	52.85%	66.16%	PK	76.05%
KZ2-sawtooth19	97.79%	PK	80.21%	91.89%	PK	86.80%
KZ2-sawtooth1	97.81%	PK	78.83%	92.20%	PK	86.98%
KZ2-venus18	97.05%	PK	78.35%	93.39%	PK	89.01%
KZ2-venus2	97.33%	PK	77.62%	93.72%	PK	89.18%
KZ2-venus6	95.36%	PK	75.06%	90.32%	PK	83.03%
lazybrush-bird	87.00%	PK	41.88%	57.42%	GR	47.32%
lazybrush-doctor	73.88%	PK	46.95%	57.40%	GR	46.96%
lazybrush-elephant	83.80%	PK	41.87%	57.35%	GR	49.69%
lazybrush-footman	81.89%	PK	58.40%	58.11%	PK	57.49%
lazybrush-hmdman	87.34%	PK	65.19%	59.86%	PK	56.53%
lazybrush-mangadinner	92.05%	PK	58.53%	55.50%	PK	52.56%
lazybrush-mangagirl	89.75%	PK	39.42%	53.04%	PK	45.75%

Continued on next page

Table A.3 (A.3 Continued)

Graph	KNFS CPU Kokkos %	KNFS CPU Hotspot	KNFS CPU %	KNFS GPU Kokkos %	KNFS GPU Hotspot	KNFS GPU %
LB07-bunny-med	81.71%	PK	67.50%	91.35%	PK	56.47%
LB07-bunny-sml	88.08%	PK	72.53%	80.91%	GR	47.34%
liver.n26c10	69.09%	PK	61.25%	77.99%	PK	49.46%
liver.n6c100	72.57%	PK	60.09%	77.64%	PK	48.49%
lux-108726-0033-009	93.64%	GR	39.15%	52.85%	PK	42.46%
lux-112498-0010-010	92.54%	PK	37.93%	61.22%	PK	62.13%
luxembourg.osm	88.03%	AK	41.17%	48.39%	PK	39.51%
nj2010-165230-0018-143	91.89%	PK	63.62%	71.43%	PK	81.15%
nj2010-165230-0032-166	83.44%	PK	49.09%	67.92%	PK	77.65%
nl-2159462-0031-021	82.64%	GR	55.08%	57.82%	GR	56.77%
nl-2159462-0276-041	78.66%	PK	43.79%	63.29%	PK	49.94%
pedestrians.inc	56.11%	PK	66.59%	83.09%	PK	83.52%
person_16bins	73.97%	PK	57.61%	86.62%	PK	74.03%
person_64bins	73.37%	PK	59.35%	78.74%	PK	56.66%
punch-eu18-05-p	92.30%	PK	55.52%	57.78%	PK	63.28%
punch-eu20-08-u	87.56%	PK	41.74%	54.95%	PK	49.48%
punch-eu20-09-p	87.51%	PK	54.21%	58.12%	PK	58.34%
punch-eu22-06-p	79.72%	PK	53.75%	58.72%	PK	49.19%
punch-us18-01-u	90.85%	PK	45.10%	53.71%	PK	56.50%
punch-us18-07-u	91.23%	PK	44.15%	53.48%	PK	57.49%
punch-us18-08-p	91.33%	PK	50.53%	57.08%	PK	62.31%
punch-us18-10-p	91.27%	PK	47.40%	57.71%	PK	61.91%
punch-us20-03-p	82.80%	PK	48.43%	57.88%	PK	58.55%
punch-us20-04-u	84.48%	PK	39.76%	55.66%	PK	51.08%
punch-us22-07-u	77.22%	GR	38.15%	56.03%	PK	41.89%
punch-us22-09-u	79.47%	GR	42.93%	55.68%	GR	44.72%
punch-us22-10-p	80.92%	PK	36.63%	57.00%	GR	44.20%
rgg17-129426-0025-390	83.22%	PK	51.57%	73.86%	PK	81.16%
rgg18-260352-0032-319	56.81%	PK	54.74%	82.48%	PK	87.77%
rmf-long.n2.3142.shuf	92.89%	PK	46.22%	58.49%	PK	63.12%
rmf-long.n3.3142.shuf	92.39%	PK	45.16%	57.85%	PK	58.21%
rmf-long.n4.3142.shuf	91.38%	PK	47.29%	59.36%	PK	54.35%
rmf-wide.n2.3141.shuf	95.27%	PK	44.43%	50.57%	PK	58.33%
rmf-wide.n3.3141.shuf	95.30%	PK	42.53%	51.85%	PK	54.60%
rmf-wide.n4.3141.shuf	95.47%	PK	39.09%	52.37%	PK	46.33%
videoSegA2	91.74%	PK	70.84%	77.07%	PK	62.65%
videoSegA2.inc	42.09%	PK	67.39%	91.62%	PK	90.83%
videoSegA	91.22%	PK	70.74%	77.60%	PK	63.66%
videoSegB2.inc	26.95%	PK	52.24%	95.86%	PK	91.51%
videoSegB	78.03%	PK	62.99%	81.45%	GR	62.91%
videoSegB.inc	25.76%	PK	49.65%	95.70%	PK	91.56%
wash-rlg-long.n1024.1163.shuf	94.21%	PK	49.89%	62.11%	PK	63.14%
wash-rlg-long.n2048.1163.shuf	92.83%	PK	42.71%	59.90%	PK	51.66%
wash-rlg-wide.n2048.1163.shuf	94.77%	PK	64.27%	67.98%	PK	74.67%

Appendix B

Code block snippets

B.1 Graph representation

```
//...
template <class DeviceType>
struct Graph
{

    using ExecutionSpace = typename DeviceType::execution_space;
    using MemorySpace = typename DeviceType::memory_space;
    using NodeIndex = int;
    using EdgeIndex = int;
    using ValueType = long long; // Must support atomic_add

    //...

    // CSR topology representation
    Kokkos::View<EdgeIndex *, DeviceType> row_map;
    Kokkos::View<NodeIndex *, DeviceType> entries;

    // Edge properties
    Kokkos::View<ValueType *, DeviceType> residual_capacity;
    Kokkos::View<EdgeIndex *, DeviceType> reverse_edge;

    // Vertex properties
    Kokkos::View<ValueType *, DeviceType> excess;
    Kokkos::View<ValueType *, DeviceType> added_excess;
    Kokkos::View<int *, DeviceType> label;
    Kokkos::View<int *, DeviceType> new_label;
    Kokkos::View<int *, DeviceType> active_phase;
    Kokkos::View<int *, DeviceType> active_iteration_mask;

    // State queues
    Kokkos::View<NodeIndex *, DeviceType> current_active;
    Kokkos::View<size_t, DeviceType> current_queue_size;
    Kokkos::View<NodeIndex *, DeviceType> next_active;
    Kokkos::View<size_t, DeviceType> next_queue_size;

    KOKKOS_INLINE_FUNCTION
    NodeIndex num_nodes() const { return excess.extent(0); }

    KOKKOS_INLINE_FUNCTION
    EdgeIndex num_edges() const { return entries.extent(0); }
};
//...
```

B.2 Graph loader

```
//...
const char *align_to_next_line(const char *ptr, const char *end)
{
    if (ptr >= end) return end;
    const void *nl = std::memchr(ptr, '\n', end - ptr);
    if (nl) return static_cast<const char *>(nl) + 1;

    return end;
};

// custom isspace to dodge locale checks
KOKKOS_INLINE_FUNCTION
bool fast_isspace(unsigned char c)
{
    return (c == ' ' || c == '\t' || c == '\n' || c == '\r' || c == '\f' || c == '\v');
};

// helper struct for processing
struct Chunk
{
    const char *begin;
    const char *end;
    size_t count;
    size_t offset;
};

// Main Loader
inline HostEdgeList parallel_load_dimacs(const std::string &filename, int &num_nodes,
    int &source, int &sink, unsigned int tc = 0)
{
    int fd = open(filename.c_str(), O_RDONLY);
    if (fd == -1) throw std::runtime_error("Error opening file: " + filename);

    struct stat sb;
    if (fstat(fd, &sb) == -1)
    {
        close(fd);
        throw std::runtime_error("Error getting file size");
    }
    size_t length = sb.st_size;

    if (length == 0)
    {
        close(fd);
        return HostEdgeList("empty", 0);
    }

    void *map_base = mmap(NULL, length, PROT_READ, MAP_PRIVATE, fd, 0);
    if (map_base == MAP_FAILED)
    {
        close(fd);
        throw std::runtime_error("mmap failed");
    }

    const char *data_start = static_cast<const char *>(map_base);
    const char *data_end = data_start + length;
    const char *edges_start_ptr = data_start;

    // Preamble Scan (Sequential)
    {
        const char *cur = data_start;
        bool preamble_done = false;

        while (cur < data_end && !preamble_done)
        {
            while (cur < data_end && fast_isspace(static_cast<unsigned char>(*cur))) cur++;

            if (cur >= data_end) break;

            char type = *cur;
            if (type == 'a')
            {
                edges_start_ptr = cur;
                preamble_done = true;
            }
            else if (type == 'p')
            {
                cur++;
                while (cur < data_end && fast_isspace(static_cast<unsigned char>(*cur)))
                    cur++;
                while (cur < data_end && !fast_isspace(static_cast<unsigned char>(*cur)))
```

```
                    cur++;
                while (cur < data_end && fast_isspace(static_cast<unsigned char>(*cur)))
                    cur++;
                std::from_chars(cur, data_end, num_nodes);
                cur = align_to_next_line(cur, data_end);
            }
            else if (type == 'n')
            {
                cur++;
                int id;
                while (cur < data_end && fast_isspace(static_cast<unsigned char>(*cur)))
                    cur++;
                auto res = std::from_chars(cur, data_end, id);
                cur = res.ptr;

                while (cur < data_end && fast_isspace(static_cast<unsigned char>(*cur)))
                    cur++;

                if (*cur == 's') source = id - 1;
                else if (*cur == 't') sink = id - 1;
                cur = align_to_next_line(cur, data_end);
            }
            else
            {
                cur = align_to_next_line(cur, data_end);
            }
        }
    }

    unsigned int num_chunks = Kokkos::DefaultHostExecutionSpace().concurrency();
    if (tc > 0) num_chunks = std::min(tc, num_chunks);
    if (num_chunks == 0) num_chunks = 2;
    std::cout << "Loader using " << num_chunks << " Kokkos Host Threads\n";

    Kokkos::View<Chunk *, Kokkos::HostSpace> chunks("chunks", num_chunks);

    const char *chunk_begin = edges_start_ptr;
    size_t remaining_size = data_end - edges_start_ptr;
    size_t chunk_size = (remaining_size > 0) ? (remaining_size / num_chunks) : 0;

    // Define chunks sequentially (fast, at most #of threads on the CPU)
    for (unsigned int i = 0; i < num_chunks; ++i)
    {
        const char *chunk_end = (i == num_chunks - 1) ? data_end : chunk_begin + chunk_size;
        if (i < num_chunks - 1)
        {
            chunk_end = align_to_next_line(chunk_end, data_end);
        }
        chunks(i) = {chunk_begin, chunk_end, 0, 0};
        chunk_begin = chunk_end;
    }

    // PASS 1: Parallel Count
    Kokkos::parallel_for("Pass1_CountEdges",
                         Kokkos::RangePolicy<Kokkos::DefaultHostExecutionSpace>(0, num_chunks),
                         [=](const int i)
                         {
                             size_t count = 0;
                             const char *cur = chunks(i).begin;
                             const char *end = chunks(i).end;

                             while (cur < end)
                             {
                                 while (cur < end && fast_isspace(static_cast<unsigned char>(*cur)))
                                     cur++;
                                 if (cur >= end) break;
                                 if (*cur == 'a') count++;
                                 cur = align_to_next_line(cur, end);
                             }
                             chunks(i).count = count;
    });
    Kokkos::fence();

    // Compute offsets sequentially
    size_t total_edges = 0;
    for (unsigned int i = 0; i < num_chunks; ++i)
    {
        chunks(i).offset = total_edges;
        total_edges += chunks(i).count;
    }

    HostEdgeList host_edges(
        Kokkos::ViewAllocateWithoutInitializing("host_raw_edges"),
        total_edges);

```

```
167     // PASS 2: Parallel Fill
168     Kokkos::parallel_for("Pass2_FillEdges",
169                          Kokkos::RangePolicy<Kokkos::DefaultHostExecutionSpace>(0, num_chunks),
170                          [=](const int i)
171                          {
172                              const char *cur = chunks(i).begin;
173                              const char *end = chunks(i).end;
174                              size_t current_idx = chunks(i).offset;
175 
176                              while (cur < end)
177                              {
178                                  while (cur < end && fast_isspace(static_cast<unsigned char>(*cur)))
179                                      cur++;
180                                  if (cur >= end) break;
181 
182                                  if (*cur == 'a')
183                                  {
184                                      cur++; // Skip 'a'
185                                      int u, v;
186                                      long long cap;
187 
188                                      // [u]
189                                      while (cur < end && fast_isspace(static_cast<unsigned char>(*cur)))
190                                          cur++;
191                                      auto res1 = std::from_chars(cur, end, u);
192                                      cur = res1.ptr;
193 
194                                      // [v]
195                                      while (cur < end && fast_isspace(static_cast<unsigned char>(*cur)))
196                                          cur++;
197                                      auto res2 = std::from_chars(cur, end, v);
198                                      cur = res2.ptr;
199 
200                                      // [cap]
201                                      while (cur < end && fast_isspace(static_cast<unsigned char>(*cur)))
202                                          cur++;
203                                      auto res3 = std::from_chars(cur, end, cap);
204                                      cur = res3.ptr;
205 
206                                      host_edges(current_idx) = {u - 1, v - 1, cap};
207                                      current_idx++;
208                                  }
209                                  cur = align_to_next_line(cur, end);
210                              }
211     });
212     Kokkos::fence();
213     // Tidy
214     if (munmap(map_base, length) == -1) std::cerr << "Warning: munmap failed\n";
215     close(fd);
216     return host_edges;
217 }
218 //...
```

B.3 Graph builder

```
//...
class GraphBuilder {
public:
    template <class DeviceType>
    static Graph<DeviceType> build_graph(
        const HostEdgeList& h_raw_edges,
        int num_nodes,
        int source_node,
        int sink_node
    ) {
        using ExecutionSpace = typename DeviceType::execution_space;
        using RangePolicy = Kokkos::RangePolicy<ExecutionSpace>;
        using TempView = Kokkos::View<TempEdge*, DeviceType>;
        using IntView = Kokkos::View<int*, DeviceType>;

        size_t input_size = h_raw_edges.extent(0);
        if (input_size == 0) throw std::runtime_error("Graph is empty.");

        // [HOST -> DEVICE]
        // Allocate device memory -- 2x the # of edges for residuals
        size_t potential_size = input_size * 2;
        TempView raw_edges(Kokkos::ViewAllocateWithoutInitializing("raw_edges_gpu"), potential_size);

        // copy the first half over
        Kokkos::deep_copy(
            Kokkos::subview(raw_edges, std::make_pair((size_t)0, input_size)),
            h_raw_edges
        );

        // [DEVICE] Graph Construction
        // Symmetrize -- add inverse of each edge (for i at i + input_size --> filling second half)
        Kokkos::parallel_for("Generate_Reverse_Edges", RangePolicy(0, input_size),
            KOKKOS_LAMBDA(const size_t i) {
                TempEdge e = raw_edges(i);
                raw_edges(i + input_size) = { e.v, e.u, 0 };
        });

        // Sort
        Kokkos::sort(raw_edges);

        // [DEVICE] Deduplicate & Compress
        IntView flags(Kokkos::ViewAllocateWithoutInitializing("edge_flags"), potential_size);
        Kokkos::parallel_for("Mark_Unique", RangePolicy(0, potential_size),
            KOKKOS_LAMBDA(const size_t i) {
                if (i == 0) {
                    flags(i) = 1;
                } else {
                    flags(i) = (raw_edges(i) != raw_edges(i - 1)) ? 1 : 0;
                }
        });

        IntView write_indices(Kokkos::ViewAllocateWithoutInitializing("write_indices"), potential_size);

        // inclusive scan, so that duplicates have same idx
        // [DEVICE]
        Kokkos::parallel_scan("Scan_Indices", RangePolicy(0, potential_size),
            KOKKOS_LAMBDA(const size_t i, int& update, const bool final) {
                update += flags(i);
                if (final) {
                    write_indices(i) = update - 1; // Convert to 0-based index
                }
        });

        int total_edges = 0;
        int last_idx = potential_size - 1;
        Kokkos::deep_copy(total_edges, Kokkos::subview(write_indices, last_idx));
        total_edges += 1; // 0-based index to count

        // Allocate fields on Device
        Graph<DeviceType> g;
        g.row_map = typename Graph<DeviceType>::RowMapType("row_map", num_nodes + 1);
        g.entries = typename Graph<DeviceType>::EntriesType("entries", total_edges);
        g.residual_capacity = typename Graph<DeviceType>::ValueViewType("residual", total_edges);
        g.reverse_edge = typename Graph<DeviceType>::IndexViewType("reverse_edge", total_edges);
        g.excess = typename Graph<DeviceType>::ValueViewType("excess", num_nodes);
        g.label = typename Graph<DeviceType>::LabelViewType("label", num_nodes);
        g.new_label = typename Graph<DeviceType>::LabelViewType("new_label", num_nodes);
        g.added_excess = typename Graph<DeviceType>::ValueViewType("added_excess", num_nodes);
        g.active_iteration_mask = typename Graph<DeviceType>::MaskViewType("active_mask", num_nodes);
        g.current_queue_size = Kokkos::View<size_t, DeviceType>("curr_q_size");
        g.next_queue_size = Kokkos::View<size_t, DeviceType>("next_q_size");
```

```
        g.current_active = typename Graph<DeviceType>::EntriesType("curr_active", num_nodes);
        g.next_active = typename Graph<DeviceType>::EntriesType("next_active", num_nodes);
        g.active_phase = typename Graph<DeviceType>::MaskViewType("active_phase", num_nodes);

        IntView compressed_u(Kokkos::ViewAllocateWithoutInitializing("compressed_u"), total_edges);
        Kokkos::deep_copy(g.residual_capacity, 0);

        // Populate
        Kokkos::parallel_for("Populate_CSR_Data", RangePolicy(0, potential_size),
            KOKKOS_LAMBDA(const size_t i) {
                int idx = write_indices(i);

                // Only FIRST thread of a duplicate group writes the topology
                if (flags(i) == 1) {
                    g.entries(idx) = raw_edges(i).v;
                    compressed_u(idx) = raw_edges(i).u;
                }
                // ALL threads add their capacity
                Kokkos::atomic_add(&g.residual_capacity(idx), raw_edges(i).capacity);
        });

        // Build Row Map
        // [DEVICE]
        Kokkos::deep_copy(g.row_map, 0);
        Kokkos::parallel_for("Histogram_Degrees", RangePolicy(0, total_edges),
            KOKKOS_LAMBDA(const size_t i) {
                int u = compressed_u(i);
                if (u < num_nodes) {
                    Kokkos::atomic_inc(&g.row_map(u + 1));
                }
        });

        Kokkos::parallel_scan("Scan_RowMap", RangePolicy(0, num_nodes + 1),
            KOKKOS_LAMBDA(const size_t i, int& update, const bool final) {
                update += g.row_map(i);
                if (final) g.row_map(i) = update;
        });

        // Link Reverse Edges
        // [DEVICE]
        Kokkos::parallel_for("Find_Reverse_Edges", RangePolicy(0, total_edges),
            KOKKOS_LAMBDA(const size_t i) {

                int u = compressed_u(i);
                int v = g.entries(i);
                int start = g.row_map(v);
                int end = g.row_map(v + 1);
                int low = start;
                int high = end - 1;
                int rev_idx = -1;

                while (low <= high) {
                    int mid = low + (high - low) / 2;
                    int neighbor = g.entries(mid);

                    if (neighbor == u) {
                        rev_idx = mid;
                        break;
                    } else if (neighbor < u) {
                        low = mid + 1;
                    } else {
                        high = mid - 1;
                    }
                }
                g.reverse_edge(i) = rev_idx;
        });
        // Init State
        Kokkos::deep_copy(g.excess, 0);
        Kokkos::deep_copy(g.label, 0);
        Kokkos::deep_copy(g.new_label, 0);
        Kokkos::deep_copy(g.added_excess, 0);
        Kokkos::deep_copy(g.active_iteration_mask, 0);
        Kokkos::deep_copy(g.current_queue_size, 0);
        Kokkos::deep_copy(g.next_queue_size, 0);
        Kokkos::deep_copy(g.active_phase, 0);

        return g;
    }
};
//...
```

B.4 Initialize algorithm

```
//...
template <class DeviceType>
void initialize_algorithm(Graph<DeviceType> &g, int s, int t, int n)
{
    using ExecutionSpace = typename DeviceType::execution_space;
    using RangePolicy = Kokkos::RangePolicy<ExecutionSpace>;
    using ValueType = typename Graph<DeviceType>::ValueType;

    int s_row_start, s_row_end;
    {
        auto s_map_subview = Kokkos::subview(g.row_map, std::make_pair(s, s + 2));
        auto h_s_map = Kokkos::create_mirror_view_and_copy(Kokkos::HostSpace(), s_map_subview);
        s_row_start = h_s_map(0);
        s_row_end = h_s_map(1);
    }

    Kokkos::deep_copy(Kokkos::subview(g.label, s), n);

    // Saturate Edges
    long long total_pushed_from_s = 0;
    Kokkos::parallel_reduce(
        "Saturate_Source_Edges",
        RangePolicy(s_row_start, s_row_end),
        KOKKOS_LAMBDA(const int &edge_idx, long long &local_pushed_flow) {

            // Get edge target and capacity
            int v = g.entries(edge_idx);
            long long cap = g.residual_capacity(edge_idx);

            // Technically not needed, as each edge is managed by 1 thread
            // and should have capacity as we just loaded the graph
            if (cap > 0) {
                // Push Flow
                g.residual_capacity(edge_idx) = 0;

                // Update Reverse: v -> s
                int rev_idx = g.reverse_edge(edge_idx);
                g.residual_capacity(rev_idx) += cap;
                // Update Excess at V
                // NOTE: -- this can be done without atomics
                // because we merged all edges u -> v into one
                // in graph building process
                g.excess(v) += cap;

                if (v != s && v != t) {
                    // Add to current queue
                    size_t q_pos = Kokkos::atomic_fetch_add(&g.current_queue_size(), 1);
                    g.current_active(q_pos) = v;

                    // Mark as active in iteration mask for start of algo
                    g.active_iteration_mask(v) = 1;
                    g.active_phase(v) = 1;
                }
                // Accumulate total flow pushed for the reduction
                local_pushed_flow += cap;
            }
        },
        total_pushed_from_s);

    Kokkos::deep_copy(Kokkos::subview(g.excess, s), -total_pushed_from_s);
    Kokkos::fence();
}
//...
```

B.5 Process kernel

```
//...
while (h_current_q_size > 0)
{
    //...
    // GR logic
    // ...
    int next_iter_mask = iteration + 1;

    Kokkos::parallel_for(
        "process_kernel",
        Kokkos::RangePolicy<Device>(0, h_current_q_size),
        KOKKOS_LAMBDA(const int &i) {
            int u = g.current_active(i);
            if (u == s || u == t) return;

            long long e_u = g.excess(u);

            // label at the moment kernel was
            // launched (this wont change)
            const int d_u_start = g.label(u);

            // current label reflecting
            // local relabeling when discharging
            int d_u_current = d_u_start;

            // edges from u
            int row_start = g.row_map(u);
            int row_end = g.row_map(u + 1);

            // We loop until discharged or blocked by a conflict
            while (e_u > 0)
            {
                // "Infinity"
                unsigned long min_d_neighbor = N + 1;
                bool skipped_admissible_edge = false;

                // go over all the edges from u and try
                // to push along them if possible
                for (int idx = row_start; idx < row_end; ++idx)
                {
                    int v = g.entries(idx);
                    long long cap = g.residual_capacity(idx);

                    // can we still push?
                    if (cap > 0)
                    {
                        int d_v = g.label(v);

                        // note min neighbour for relabeling later
                        if (d_v < min_d_neighbor) min_d_neighbor = d_v;

                        // admissible ?
                        if (d_u_current == d_v + 1)
                        {
                            bool wins = true;
                            // are we contesting this edge?
                            if (g.active_phase(v) == iteration)
                            {
                                wins =  (d_u_start < d_v - 1) ||
                                        (d_u_start == d_v + 1) ||
                                        (d_u_start == d_v && u < v);
                            }

                            if (wins)
                            {
                                // u won, so u can now safely
                                // discharge along edge idx
                                long long delta = (e_u < cap) ? e_u : cap;

                                g.residual_capacity(idx) -= delta;
                                g.residual_capacity(g.reverse_edge(idx)) += delta;
                                e_u -= delta;
                                // this has to be atomic as u' might exist
                                // also pushing into v at the same time
                                Kokkos::atomic_add(&g.added_excess(v), delta);

                                // Enqueue v
                                if (v != s && v != t)
                                {
                                    // mark vertices active in NEXT turn
                                    int seen_mask = Kokkos::atomic_exchange(&g.active_iteration_mask(v), next_iter_mask);
```

```
                                    if (seen_mask != next_iter_mask)
                                    {
                                        size_t insert_pos = Kokkos::atomic_fetch_add(&g.next_queue_size(), 1);
                                        g.next_active(insert_pos) = v;
                                    }
                                }

                                // did we just push all excess of u?
                                // if so, break now from the for loop
                                if (e_u == 0) break;
                            }
                            // we lost - wait it out
                            else
                            {
                                // Found admissible edge, but lost conflict.
                                skipped_admissible_edge = true;
                                continue;
                            }
                        } // not admissible
                    } // cap <= 0
                } // end of edge loop

                // break from while loop
                // if we have nothing left to push
                if (e_u == 0) break;

                // the moment we lost the win check
                // we must break to not relabel
                // as we could skip the edge for good
                if (skipped_admissible_edge) break;

                // we are here - we have excess and did not skip adm. edges
                // -> relabel and re-scan edges
                int new_d = min_d_neighbor + 1;

                // apply relabel if valid and increasing
                if (new_d < N + 1 && new_d > d_u_start)
                {
                    d_u_current = new_d;
                    g.new_label(u) = new_d;
                }
                else
                {
                    // If we can't push and can't relabel (e.g. disconnected), we are stuck.
                    break;
                }
            } // no excess/skipped adm

            // Write back remaining excess
            g.excess(u) = e_u;

            // enqueue Self if still active / relabeled
            if (e_u > 0 || d_u_current > d_u_start)
            {
                int seen_mask = Kokkos::atomic_exchange(&g.active_iteration_mask(u), next_iter_mask);
                if (seen_mask != next_iter_mask)
                {
                    size_t insert_pos = Kokkos::atomic_fetch_add(&g.next_queue_size(), 1);
                    g.next_active(insert_pos) = u;
                }
            }
    });

    Kokkos::fence("after_process_fence");
    // update work metric
    iterations_since_last_gr++;
//...
// Apply kernel...
//...
}
```

B.6 Apply kernel

```
//...
if (h_next_q_size > 0)
{
    Kokkos::parallel_for(
        "apply_kernel",
        Kokkos::RangePolicy<Device>(0, h_next_q_size),
        KOKKOS_LAMBDA(const int &i) {
            int u = g.next_active(i);
            long long incoming = g.added_excess(u);
            // update excess added by process
            // kernel prior, so that excess
            // is up-to-date for next iter
            if (incoming > 0 || g.excess(u))
            {
                g.excess(u) += incoming;
                g.added_excess(u) = 0;
                g.active_phase(u) = next_iter_mask;
            }
            int d_proposed = g.new_label(u);
            int d_current = g.label(u);
            // if we relabeled during the process
            // kernel, update
            if (d_proposed > d_current)
            {
                g.label(u) = d_proposed;
                g.new_label(u) = 0;
            }
    });
    Kokkos::fence("after_apply_fence");
}
//...
```

B.7 Global relabel

```
//...
Kokkos::parallel_for("GlobalRelabel_Fused_Init",
    RangePolicy(0, n), KOKKOS_LAMBDA(const int v)
    {
        if (v == t) {
            // This is the Sink
            g.label(v) = 0;

            // Initialize BFS Queue with Sink
            g.current_active(0) = t;
            g.current_queue_size() = 1;
        }
        else if(v == s){
            g.label(v) = n;
        }
        else {
            // This is a normal node
            g.label(v) = n+1;
        }
    });
// Fence to ensure the queue is ready before the Host reads size in the loop
Kokkos::fence();

// Run until queue is empty or we exceed N (in N steps we must reach all N nodes...duh)
size_t host_current_q_size = 1;
for (int dist = 0; dist < n; ++dist)
{
    // Break condition
    if (host_current_q_size == 0) break;

    size_t host_next_q_size = 0;

    Kokkos::parallel_reduce("GlobalRelabel_BFS_Expand",
        RangePolicy(0, host_current_q_size),
        KOKKOS_LAMBDA(const int q_idx, size_t &l_added)
        { // l_added is the thread-local accumulator
            int u = g.current_active(q_idx);
            int start = g.row_map(u);
            int end = g.row_map(u + 1);

            for (int i = start; i < end; ++i)
            {
                int v = g.entries(i);
                // skip s
                if (v == s) continue;

                int rev_idx = g.reverse_edge(i);
                if (g.residual_capacity(rev_idx) > 0)
                {
                    int expected_label = n+1;
                    int new_label_val = dist + 1;

                    if (Kokkos::atomic_compare_exchange(&g.label(v), expected_label, new_label_val) == expected_label)
                    {
                        int next_pos = Kokkos::atomic_fetch_add(&g.next_queue_size(), 1);
                        g.next_active(next_pos) = v;
                        l_added++;
                    }
                }
            }
        },
        host_next_q_size); // Reduce into host_next_q_size

    Kokkos::fence();

    host_current_q_size = host_next_q_size;

    // Swap Queues
    std::swap(g.current_active, g.next_active);
    std::swap(g.current_queue_size, g.next_queue_size);

    // zero out the device (GPU) counter so the atomics start at 0 next iteration.
    Kokkos::deep_copy(g.next_queue_size, 0);
}
//...
```

Bibliography

1. DANTZIG, George B.; FULKERSON, Delbert Ray. On the Max Flow Min Cut Theorem of Networks. In: 1955. Available also from: https://web.archive.org/web/20180505093256/http://www.dtic.mil/dtic/tr/fulltext/u2/605014.pdf.
2. HARRIS, T. E.; ROSS, Susan D. Fundamentals of a Method for Evaluating Rail Net Capacities. In: 1955. Available also from: https://web.archive.org/web/20140108021700/http://www.dtic.mil/dtic/tr/fulltext/u2/093458.pdf.
3. MUKHERJEE, Taniya; SANGAL, Isha; SARKAR, Biswajit; ALKADASH, Tamer M. Mathematical estimation for maximum flow of goods within a cross-dock to reduce inventory. *Mathematical Biosciences and Engineering*. 2022, vol. 19, no. 12, pp. 13710–13731. Available from DOI: 10.3934/mbe.2022639.
4. BOYKOV, Y.; KOLMOGOROV, V. An experimental comparison of min-cut/max- flow algorithms for energy minimization in vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2004, vol. 26, no. 9, pp. 1124–1137. Available from DOI: 10.1109/TPAMI.2004.60.
5. RANJINI, Paul Sheeba; HEMAMALINI, Tulasiram; SENTHILVEL, Angalakurichi Natraj. Optimizing Network Load Distribution With Maximum Flow Algorithms In The WSCLB Framework. *International Journal of Environmental Sciences*. 2025, pp. 1430–1440. Available from DOI: 10.64252/6tk68116.
6. CHEN, Li; KYNG, Rasmus; LIU, Yang P.; PENG, Richard; GUTENBERG, Maximilian Probst; SACHDEVA, Sushant. Almost-Linear-Time Algorithms for Maximum Flow and Minimum-Cost Flow. *Commun. ACM*. 2023, vol. 66, no. 12, pp. 85–92. ISSN 0001-0782. Available from DOI: 10.1145/3610940.

7. FORD, L. R.; FULKERSON, D. R. Maximal Flow Through a Network. *Canadian Journal of Mathematics*. 1956, vol. 8, pp. 399–404. Available from DOI: 10.4153/CJM-1956-045-5.

8. EDMONDS, Jack; KARP, Richard M. Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems. *J. ACM*. 1972, vol. 19, no. 2, pp. 248–264. ISSN 0004-5411. Available from DOI: 10.1145/321694.321699.

9. GOLDBERG, Andrew V.; TARJAN, Robert E. A new approach to the maximum-flow problem. *J. ACM*. 1988, vol. 35, no. 4, pp. 921–940. ISSN 0004-5411. Available from DOI: 10.1145/48014.61051.

10. VALIANT, L G. A bridging model for parallel computation. *Communications of the ACM (Association of Computing Machinery); (USA)*. 1990, vol. 33:8. ISSN ISSN 0001-0782. Available from DOI: 10.1145/79173.79181.

11. BAUMSTARK, Niklas; BLELLOCH, Guy; SHUN, Julian. *Efficient Implementation of a Synchronous Parallel Push-Relabel Algorithm*. 2015. Available from arXiv: 1507.01926.

12. VOLKOV, Vasily. Understanding Latency Hiding on GPUs. In: 2016. Available also from: https://api.semanticscholar.org/CorpusID:57827291.

13. NVIDIA. *CUDA C++ Programming Guide*. Nvidia, 2026. Available also from: https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf.

14. ARTIGUES, Victor; KORMANN, Katharina; RAMPP, Markus; REUTER, Klaus. *Evaluation of performance portability frameworks for the implementation of a particle-in-cell code*. 2019. Available from arXiv: 1911.08394.

15. DAMERUPPULA, Suresh. The evolution and impact of high-performance computing systems. *Journal Of Computer Networks Architecture and High Performance Computing*. 2025, pp. 1983–1989. Available from DOI: 10.30574/wjarr.2025.26.1.1247.

16. MORGAN, Timothy Prickett. *Modest HPC centers drive TOP500 supercomputer rankings this time around* [The Next Platform]. 2025. Available also from: https://www.nextplatform.com/hpc/2025/11/17/modest-hpc-centers-drive-top500-supercomputer-rankings-this-time-around/1696804.

17. TROTT, Christian R.; LEBRUN-GRANDIÉ, Damien; ARNDT, Daniel; CIESKO, Jan; DANG, Vinh; ELLINGWOOD, Nathan; GAYATRI, Rahulkumar; HARVEY, Evan; HOLLMAN, Daisy S.; IBANEZ, Dan; LIBER, Nevin; MADSEN, Jonathan; MILES, Jeff; POLIAKOFF, David; POWELL, Amy; RAJAMANICKAM, Sivasankaran; SIMBERG, Mikael; SUN-

DERLAND, Dan; TURCKSIN, Bruno; WILKE, Jeremiah. Kokkos 3: Programming Model Extensions for the Exascale Era. *IEEE Transactions on Parallel and Distributed Systems*. 2022, vol. 33, no. 4, pp. 805–817. Available from DOI: 10.1109/TPDS.2021.3097283.

18. EDWARDS, H. Carter; TROTT, Christian R.; SUNDERLAND, Daniel. Kokkos: Enabling manycore performance portability through polymorphic memory access patterns. *Journal of Parallel and Distributed Computing*. 2014, vol. 74, no. 12, pp. 3202–3216. ISSN 0743-7315. Available from DOI: 10.1016/j.jpdc.2014.07.003.

19. THE KOKKOS TEAM. *Kokkos Core Wiki Source Repository* [https://github.com/kokkos/kokkos-core-wiki]. GitHub, 2026.

20. KOKKOS TEAM. *Kokkos Tutorials: Tutorials for the Kokkos C++ Performance Portability Programming Ecosystem* [https://github.com/kokkos/kokkos-tutorials]. GitHub, 2026.

21. CHEN, Li; KYNG, Rasmus; LIU, Yang P.; PENG, Richard; GUTENBERG, Maximilian Probst; SACHDEVA, Sushant. *Maximum Flow and Minimum-Cost Flow in Almost-Linear Time*. 2022. Available from arXiv: 2203.00671 [cs.DS].

22. HOCHBAUM, Dorit S. The Pseudoflow Algorithm: A New Algorithm for the Maximum-Flow Problem. *Oper. Res.* 2008, vol. 56, no. 4, pp. 992–1009. ISSN 0030-364X. Available from DOI: 10.1287/opre.1080.0524.

23. CHANDRAN, Bala; HOCHBAUM, Dorit. A Computational Study of the Pseudoflow and Push-Relabel Algorithms for the Maximum Flow Problem. *Operations Research*. 2009, vol. 57, pp. 358–376. Available from DOI: 10.1287/opre.1080.0572.

24. HOCHBAUM, Dorit S. *HPF - Hochbaum's PseudoFlow Solver* [https://riot.ieor.berkeley.edu/Applications/Pseudoflow/maxflow.html]. 2009. Accessed: 2026-03-07.

25. CHERKASSKY, B. V.; GOLDBERG, A. V. On Implementing the Push—Relabel Method for the Maximum Flow Problem. *Algorithmica*. 1997, vol. 19, no. 4, pp. 390–410. ISSN 1432-0541. Available from DOI: 10.1007/PL00009180.

26. 13TH DIMACS IMPLEMENTATION CHALLENGE ORGANIZERS. *Maxflow reference solvers* [https://coral.ise.lehigh.edu/flow-challenge-2-0/max-flow-reference-solvers/]. 2026. Accessed: 2026-03-07.

27. VANAUSDAL, Avery; BURTSCHER, Martin. An Efficient Push-Relabel Implementation for Max-Flow Computations on GPUs. In: 2025, pp. 1–10. Available from DOI: 10.1109/IPCCC66453.2025.11304666.

28. BURTSCHER, Martin; VANAUSDAL, Avery. *ECL-MaxFlow: An efficient CUDA implementation of the Push-Relabel algorithm* [https://github.com/burtscher/ECL-MaxFlow/]. GitHub, 2026.

29. BAUMSTARK, Niklas; BLELLOCH, Guy; SHUN, Julian. *Reference implementation of PRSN algorithm* [https://github.com/niklasb/pbbs-maxflow/tree/master]. GitHub, 2016.

30. CICHRA, Radek. *fit-dp* [https://github.com/cichrrad/fit-dp]. 2026. Main.

31. SVEIDQVIST, Knut; MERMAID CONTRIBUTORS. *Mermaid: Generation of diagrams and flowcharts from text in a similar manner as markdown* [https://mermaid.js.org]. GitHub, 2026. Rendered via the Mermaid Live Editor (https://mermaid.live/).

32. WATERLOO COMPUTER VISION GROUP, University of. *Max-flow problem instances in vision* [https://vision.cs.uwaterloo.ca/data/maxflow]. [N.d.].

33. BEAMER, Scott; ASANOVIĆ, Krste; PATTERSON, David. Direction-Optimizing Breadth-First Search. *Scientific Programming*. 2013, vol. 21. Available from DOI: 10.3233/SPR-130370.

34. CUTHILL, E.; MCKEE, J. Reducing the bandwidth of sparse symmetric matrices. In: *Proceedings of the 1969 24th National Conference*. New York, NY, USA: Association for Computing Machinery, 1969, pp. 157–172. ACM '69. ISBN 9781450374934. Available from DOI: 10.1145/800195.805928.

35. BOYKOV, Yuri; KOLMOGOROV, Vladimir. An Experimental Comparison of Min-Cut/Max-Flow Algorithms for Energy Minimization in Vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2004, vol. 26, pp. 1124–1137. Available also from: http://www.csd.uwo.ca/~yuri/Abstracts/pami04-abs.shtml.